

이산수학 · 5장

관계

Relations

미리보기 — 관계란 무엇인가

관계는 원소와 원소 사이에 맺어지는 연결이다. 일상에서 우리는 끊임없이 관계를 다룬다. '누가 누구의 친구인가', '어떤 수가 어떤 수로 나누어떨어지는가', '어느 도시에서 어느 도시로 갈 수 있는가'가 모두 관계이다.

핵심 한 줄

관계란 '순서쌍들의 집합'이다. 두 대상이 관계를 맺는다는 것은, 그 둘로 이루어진 순서쌍이 어떤 집합에 들어 있다는 뜻이다.

AI 응용 — 왜 지금 관계를 배우는가

현대 인공지능의 핵심 자료구조는 대부분 관계이다. 소셜 네트워크의 친구 그래프, 추천 시스템의 사용자-상품 행렬, 지식 그래프(knowledge graph)의 (주어, 술어, 목적어) 삼항관계, 그리고 RAG 시스템이 문서를 검색할 때 쓰는 '질의-문서 유사도 관계'가 그러하다. 5장에서 배우는 관계행렬과 경로 개념은 이 모든 시스템의 수학적 토대이다.

이 장의 학습 목표

- 1 곱집합과 순서쌍의 개념을 이해하고 곱집합의 크기를 계산한다.
- 2 이항관계의 정의를 알고, 화살표 그림 · 관계행렬 · 유향 그래프 세 가지로 표현한다.
- 3 관계행렬의 부울곱으로 경로와 연결관계를 계산한다.
- 4 반사 · 대칭 · 추이 등 관계의 성질을 판별하고, 동치관계와 동치류를 구한다.
- 5 역관계 · 여관계 · 합성관계를 정의하고 계산한다.
- 6 와샬 알고리즘으로 연결관계(추이폐포)를 유한 번에 구한다.
- 7 각 개념을 Lean 4 형식 증명으로 1:1 대응시켜 엄밀하게 확인한다.

목차

5.1

곱집합

순서쌍과 곱집합, 곱집합의 크기

5.2

관계와 관계 표현

이항관계, 화살표 그림 · 관계행렬 · 유향 그래프

5.3

경로

경로와 순환, 연결관계, 부울곱

5.4

관계의 성질

반사 · 대칭 · 추이, 동치관계와 동치류

5.5

역관계와 합성관계

역관계, 여관계, 합성관계

5.6

연결관계와 와샬 알고리즘

폐포, 와샬 알고리즘

PART

5.1

곱집합

순서쌍과 곱집합, 그리고 곱집합의 크기를 다룬다.
관계를 정의하기 위한 첫 번째 재료이다.

순서쌍 — 순서가 있는 묶음

순서쌍(ordered pair)은 두 대상을 '순서를 정하여' 묶은 것이다. 첫 번째 자리와 두 번째 자리가 구별되므로, (a, b) 와 (b, a) 는 서로 다르다. 이것이 집합 $\{a, b\}$ 와의 결정적 차이이다.

정의 — 순서쌍

세 개의 대상을 순서대로 묶은 (a, b, c) 를 순서삼중쌍이라 한다. 일반적으로 n 개를 순서대로 묶은 $(a_1, a_2, \dots, a_n)$ 을 순서 n 중쌍(n -tuple)이라 한다. i 번째 자리의 원소를 i 번째 성분이라 부른다.

흔한 오개념

집합 $\{1, 2\}$ 와 $\{2, 1\}$ 은 같지만, 순서쌍 $(1, 2)$ 와 $(2, 1)$ 은 다르다. 순서쌍에서는 '자리'가 의미를 가진다.

AI 연결

딥러닝의 벡터 $(x_1, x_2, \dots, x_n)$ 가 바로 순서 n 중쌍이다. 각 성분의 자리(차원)가 고정된 의미를 가지므로, 성분의 순서를 바꾸면 다른 데이터가 된다.

순서쌍의 상등

두 순서쌍이 같으려면 같은 자리의 성분이 모두 같아야 한다. 이 조건이 순서쌍 상등의 정의이다.

순서쌍 상등의 조건

$$(a, b) = (c, d) \iff a = c \wedge b = d$$

이 조건은 좌변에서 우변으로(같으면 성분이 같다), 우변에서 좌변으로(성분이 같으면 같다) 모두 성립하는 필요충분조건이다. 그래서 양방향 화살표를 사용한다.

필요충분조건의 의미를 미리

왼쪽 화살표만 성립하면 '~이면(if)', 양방향이면 '~일 필요충분조건(if and only if)'이다. 이 차이는 5.4절 관계의 성질과 Lean 4 증명 전반에서 거듭 등장하므로 지금 정확히 기억해 두는 것이 좋다.

예제 5-1 — 일상 속 순서쌍

문제

생일은 태어난 달과 일의 순서쌍으로 나타낼 수 있고, 학생은 학년 · 반 · 번호로 나타낼 수 있다. 어떤 학생의 생일이 12월 25일이고 3학년 4반 27번일 때, 각각을 순서쌍으로 표현하라.

증명

생일은 (달, 일) 의 순서쌍이므로 (12, 25) 이다.

학년 · 반 · 번호는 순서삼중쌍이므로 (3, 4, 27) 이다.

두 순서쌍이 같을 필요충분조건은 같은 자리의 성분이 각각 서로 같은 것이다. 예컨대 (12, 25) 는 (25, 12) 과 다르다 — 달과 일의 자리가 바뀌었기 때문이다.

STUDENT TRY

여러분의 학번을 순서쌍으로 표현해 보라. 입학년도 · 학과코드 · 일련번호의 세 성분으로 나눌 수 있는가? 자리를 바꾸면 왜 의미가 달라지는지 한 문장으로 설명해 보라.

예제 5-2 — 순서쌍 상등으로 미지수 구하기

문제

다음 두 순서쌍이 같기 위한 x 와 y 의 값을 구하라. $(2x + 1, 5) = (7, y + 3)$

증명

순서쌍이 같으려면 같은 자리의 성분이 각각 같아야 한다.

$$\text{첫 번째 성분: } 2x + 1 = 7 \quad \Rightarrow \quad 2x = 6 \quad \Rightarrow \quad x = 3$$

$$\text{두 번째 성분: } 5 = y + 3 \quad \Rightarrow \quad y = 2$$

따라서 $x = 3, y = 2$ 이다.

STUDENT TRY

$(a - 1, 2b) = (4, 10)$ 을 만족하는 a, b 를 구하라. 풀이의 첫 줄에 반드시 '성분이 각각 같다'를 적을 것.

곱집합 — 모든 순서쌍을 모은 집합

정의 곱집합

곱집합 cartesian product $A \times B$ 는 공집합이 아닌 임의의 두 집합 A , B 에 대해 $a \in A$ 이고 $b \in B$ 인 모든 순서쌍 (a, b) 들의 집합을 말한다.

$$A \times B = \{(a, b) \mid a \in A \text{이고 } b \in B\}$$

곱집합 $A \times B$ 는 'A에서 첫 성분을 하나, B에서 두 번째 성분을 하나 뽑아 만든 모든 순서쌍'의 집합이다. A와 B의 역할이 다르므로, 일반적으로 $A \times B$ 와 $B \times A$ 는 서로 다르다.

주의

곱집합의 '곱($\times$)'은 숫자 곱셈이 아니다. 그러나 다음 절에서 보듯 크기를 따지면 곱셈과 정확히 닮아 있다.

예제— $A \times B$ 와 $B \times A$

문제 $A = \{1, 2, 3, 4\}$ 이고 $B = \{x, y\}$ 일 때, 곱집합 $A \times B$ 와 $B \times A$ 를 모두 구하라.

$A \times B$ 는 A 의 원소를 첫 자리에, B 의 원소를 둘째 자리에 놓는다. 원소가 4 개와 2개이므로 순서쌍은 $4 \times 2 = 8$ 개가 나온다.

풀이

$A \times B$ 는 A 에서 첫 번째 원소를 꺼내고, B 에서 두 번째 원소를 꺼내 만든 모든 순서쌍들의 집합이다.

$$A \times B = \{(1, x), (1, y), (2, x), (2, y), (3, x), (3, y), (4, x), (4, y)\}$$

마찬가지로 $B \times A$ 는 B 에서 첫 번째 원소를 꺼내고, A 에서 두 번째 원소를 꺼내 만든 모든 순서쌍들의 집합이다.

$$B \times A = \{(x, 1), (x, 2), (x, 3), (x, 4), (y, 1), (y, 2), (y, 3), (y, 4)\}$$

곱집합의 확장 — n개의 집합

곱집합은 두 집합뿐 아니라 임의의 n개 집합으로 확장된다. 각 집합에서 성분을 하나씩 뽑아 순서 n중쌍을 만든다.

정의 — n개 집합의 곱집합

$$A_1 \times A_2 \times \cdots \times A_n = \{ (a_1, a_2, \dots, a_n) : a_i \in A_i \}$$

AI 응용 — 곱집합과 특징 공간

기계학습에서 데이터 한 건은 여러 특징(feature)의 순서 n중쌍이다. 예를 들어 집값 예측 데이터는 (면적, 방 개수, 지하철 거리, 건축 연도) 처럼 표현된다. 모든 가능한 데이터의 모음이 곧 특징 집합들의 곱집합이며, 이를 특징 공간(feature space)이라 부른다. RAG 시스템에서 문서를 표현하는 임베딩 벡터도 수백 차원 집합들의 곱집합 위의 한 점이다.

예제 5-4 — 세 집합의 곱집합

문제 한 소프트웨어 회사가 언어 $L = \{C, P\}$, 메모리 $M = \{256, 512\}$, 운영체제 $O = \{U, W\}$ 를 제공한다. 곱집합 $L \times M \times O$ 를 모두 나열하고 원소의 개수를 구하라.

L 에서 첫 성분, M 에서 둘째 성분, O 에서 셋째 성분을 뽑아 순서삼중쌍을 만든다. 각각 2개씩이므로 $2 \times 2 \times 2 = 8$ 개가 예상된다.

$L \times M \times O$ 는 L 에서 첫 번째 원소를, M 에서 두 번째 원소를, O 에서 세 번째 원소를 꺼내 만든 3개의 원소를 가진 모든 순서쌍들의 집합이다.

$L \times M \times O = \{(C, 256, U), (C, 256, W), (C, 512, U), (C, 512, W), (P, 256, U), (P, 256, W), (P, 512, U), (P, 512, W)\}$ 이다.

따라서 곱집합 $L \times M \times O$ 의 원소의 개수는 8개이다.

정리 5-1 — 곱집합의 크기

정리 5-1 곱집합의 크기

A 와 B 를 공집합이 아닌 유한집합이라고 할 때, A 와 B 의 곱집합의 크기^{cardinality}는 $|A \times B| = |A| \times |B|$ 이다.

세 집합으로 확장하면 $|A \times B \times C| = |A| \cdot |B| \cdot |C|$ 이고, 일반적으로 n 개 집합에 대해 다음이 성립한다.

n 개 집합으로 확장

$$|A_1 \times A_2 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$$

정리 5-1 — 증명 (곱의 법칙)

증명 두 집합의 경우를 증명한다. $|A| = m$, $|B| = n$ 이라 하자.

(1단계) $A \times B$ 의 순서쌍 (a, b) 를 만드는 절차는 두 단계로 나뉜다.
첫째 성분 a 를 고르는 일, 둘째 성분 b 를 고르는 일이다.

(2단계) 첫째 성분 a 는 A 의 원소 중 하나이므로 고르는 방법이 m 가지이다.
그 각각에 대해 둘째 성분 b 는 B 의 원소 중 하나이므로 n 가지이다.

(3단계) 곱의 법칙에 의해 (a, b) 를 만드는 방법은 모두 $m \cdot n$ 가지이다.
서로 다른 절차는 서로 다른 순서쌍을 낳으므로 $|A \times B| = m \cdot n = |A| \cdot |B|$. ■

n 개 집합으로의 확장은 수학적 귀납법을 쓴다..

STUDENT TRY

n 개 집합으로 수학적 귀납법을 이용하여 증명 해보자.

예제 5-5 — 곱집합의 크기 계산

문제

정리 5-1 을 이용해 예제 5-3 의 $A \times B$ 와 예제 5-4 의 $L \times M \times O$ 의 크기를 구하라.

증명

예제 5-3: $|A \times B| = |A| \cdot |B| = 4 \cdot 2 = 8$

예제 5-4: $|L \times M \times O| = |L| \cdot |M| \cdot |O| = 2 \cdot 2 \cdot 2 = 8$

앞서 직접 나열하여 센 결과 (각각 8개) 와 정확히 일치한다.

직접 나열은 원소가 많아지면 비현실적이다. 정리 5-1 은 나열 없이 크기만 빠르게 알려주는 도구이다.

Lean 4 — 곱 타입과 순서쌍

Lean 4 에서 두 타입 A, B 의 곱집합은 곱 타입 $A \times B$ 로 표현된다. 순서쌍 (a, b) 는 이 타입의 값이다.

```
-- A × B 는 곱 타입. (a, b) 는 그 타입의 값이다.
```

```
def p : Nat × Nat := (3, 5)
```

lean

```
-- .1 은 첫째 성분, .2 는 둘째 성분을 꺼낸다.
```

```
example : p.1 = 3 := rfl
```

```
example : p.2 = 5 := rfl
```

```
-- 곱 타입은 세 개 이상으로도 중첩된다: Nat × Nat × Nat
```

```
def t : Nat × Nat × Nat := (3, 4, 27)
```

rfl 은 'reflexivity', 곧 양변이 계산하면 글자 그대로 같다는 뜻이다. p.1 은 계산하면 3 이 되므로 rfl 로 증명된다.

수학과의 대응

수학의 '순서쌍 (a, b) ' 가 Lean 의 ' $(a, b) : A \times B$ ' 와 글자 그대로 대응한다. 수학의 첫째 · 둘째 성분 이 Lean 의 .1 과 .2 이다.

수학 ↔ Lean — 순서쌍 상등 정리

수학 진술

증명

주장:

$$(a, b) = (c, d) \\ \Leftrightarrow a = c \text{ 그리고 } b = d$$

증명 (개념):

순서쌍의 상등은 '같은 자리의
성분이 모두 같음' 으로
정의된다. 이 정의 자체가
곧 필요충분조건이다.

Lean 에서는 이 사실이
Prod.mk.injEq 라는
이름의 정리로 들어 있다.

Lean 4 증명

lean

```
-- 순서쌍 상등: Prod.mk.injEq
example (a c b d : Nat) :
  ((a, b) = (c, d))
    ⇔ (a = c ∧ b = d) := by
  -- 정의에 해당하는 정리로 재서술
  rw [Prod.mk.injEq]

-- 화살표 ⇔ 는 '필요충분조건'.
-- rw 한 번으로 목표가
-- 정확히 정리의 진술과
-- 일치하여 증명이 닫힌다.
```

수학의 '정의' 가 Lean 에서는 '이미 증명된 정리
(Prod.mk.injEq)' 로 제공된다. rw 는 목표를 그 정리로 바꿔 쓴
다.

Lean 4 — 예제 5-2 를 형식 증명으로

예제 5-2 의 $(2x+1, 5) = (7, y+3)$ 을 Lean 으로 옮긴다. 순서쌍 상등으로 성분별 등식을 얻고, 산술로 마무리한다.

```
example (x y : Int)
  (h : ((2*x + 1, (5:Int))) = (7, y + 3))) :
  x = 3 ∧ y = 2 := by
  -- (1) 순서쌍 상등으로 성분별 등식 두 개를 얻는다
  rw [Prod.mk.injEq] at h
  -- 이제 h : 2*x + 1 = 7 ∧ 5 = y + 3
  obtain ⟨h1, h2⟩ := h -- 두 등식으로 분해
  -- (2) 각 성분의 결론을 산술로 닫는다
  constructor
  · omega -- 2*x+1=7 ⇒ x=3
  · omega -- 5=y+3 ⇒ y=2
```

lean

`obtain ⟨h1, h2⟩` 은 'A 그리고 B' 형태의 가정을 둘로 쪼갬다. `constructor` 는 'A 그리고 B' 형태의 목표를 둘로 쪼갬다.

Python — 곱집합을 코드로

곱집합을 빠짐없이 나열하는 작업은 Python 의 `itertools.product` 로 한 줄에 끝난다. 예제 5-4 의 $L \times M \times O$ 를 직접 만들어 본다.

python

```
from itertools import product

L = ['C', 'P']
M = [256, 512]
O = ['U', 'W']

LMO = list(product(L, M, O))    # 곱집합  $L \times M \times O$ 
print(LMO)                      # 8개의 순서삼중쌍
print(len(LMO))                 #  $8 = |L| \cdot |M| \cdot |O| = 2 \cdot 2 \cdot 2$ 
```

5.1 요약 — 곱집합

순서쌍

순서가 있는 묶음. $(a,b) \neq (b,a)$ 가 일반적.

순서쌍 상등

같은 자리의 성분이 모두 같을 필요충분조건.

곱집합

$A \times B$ = 각 집합에서 성분을 뽑은 모든 순서쌍.

곱집합의 크기

$|A \times B| = |A| \cdot |B|$ (곱의 법칙).

Lean 대응

곱 타입 $A \times B$, 정리 `Prod.mk.injEq`.

PART

5.2

관계와 관계 표현

이항관계의 정의와, 화살표 그림 · 관계행렬 · 유향 그래프
세 가지 표현 방법을 익힌다.

이항관계 — 곱집합의 부분집합

곱집합 $A \times B$ 는 두 집합의 원소로 만들 수 있는 '모든' 순서쌍의 모임이다. 그중 우리가 관심 있는 순서쌍만 골라내면 그것이 바로 관계이다.

정의 5-3 이항관계

두 집합 A, B 에 대해 집합 A 에서 집합 B 로 가는 관계를 **이항관계** binary relation라고 한다. 이항관계 R 은 $A \times B$ 의 부분집합들이다. $x \in A$ 와 $y \in B$ 에 대해 $(x, y) \in R$ 이면 x 는 y 에 대해 R 의 관계에 있다고 하고, $x R y$ 로 표기한다. 또한 $(x, y) \notin R$ 이면 x 는 y 에 대해 R 의 관계가 없다고 하고, $x \not R y$ 로 표기한다.

$$R \subseteq A \times B$$

집합 A 에서 집합 B 로의 이항관계 R 은 곱집합 $A \times B$ 의 부분집합이다. 곧 ' $A \times B$ 의 순서쌍 중 일부를 골라 모은 집합'이 관계이다.

관계 표기

$$a R b \iff (a, b) \in R$$

순서쌍 (a, b) 가 R 에 속하면 ' a 와 b 가 관계 R 에 있다'고 하며 $a R b$ 로 쓴다.

예제 5-6 — 이항관계인지 확인

문제

$A = \{1, 2\}$, $B = \{a, b, c\}$ 이고 $R = \{(1,a), (1,b), (2,b), (2,c)\}$ 일 때, R 이 A 에서 B 로의 이항관계인지 확인하고 원소 간 관계를 표기하라.

증명

R 이 이항관계이려면 R 이 $A \times B$ 의 부분집합이어야 한다.

$$A \times B = \{(1,a), (1,b), (1,c), (2,a), (2,b), (2,c)\}$$

R 의 네 순서쌍 $(1,a), (1,b), (2,b), (2,c)$ 가 모두 $A \times B$ 에 들어 있다.
따라서 $R \subseteq A \times B$ 이므로 R 은 A 에서 B 로의 이항관계이다.

STUDENT TRY

$S = \{(1,a), (2,d)\}$ 는 위 A, B 에 대해 이항관계인가? 아니라면 어느 순서쌍이 문제인지 짚어 보라.

정의역 · 치역 · 공역

관계 R 의 순서쌍을 보면 첫 성분들의 모임과 둘째 성분들의 모임을 따로 생각할 수 있다.

정의역과 치역

$$\text{Dom}(R) = \{a : (a, b) \in R\}, \quad \text{Ran}(R) = \{b : (a, b) \in R\}$$

❖ 집합 A 에서 집합 B 로 가는 이항관계 R

- 관계 R 의 **정의역**domain :

R 에 속한 순서쌍에서 모든 첫 번째 원소의 집합이고 $\text{Dom}(R)$ 로 표기

- 관계 R 의 **치역**range :

R 에 속한 순서쌍에서 모든 두 번째 원소의 집합이고 $\text{Ran}(R)$ 로 표기

- 관계 R 의 **공역**codomain :

B 집합의 전체 원소로서 치역은 공역의 부분집합이 된다.

❖ 집합 A 에 관한 관계

- 만일 $A = B$ 이면, 즉 R 이 집합 A 에서 집합 A 로 가는 관계($R \subseteq A \times A$)이면 R 는 **집합 A 에 관한 관계**라 한다.

예제 5-7 — 작거나 같음 관계

문제

$A = \{1, 2, 3, 4\}$ 에서 $a R b \Leftrightarrow a \leq b$ 로 정의할 때, 관계 R 과 그 정의역 · 치역 · 공역을 구하라.

증명

$a \leq b$ 를 만족하는 순서쌍 (a, b) 를 모두 모은다.

$$R = \{(1,1), (1,2), (1,3), (1,4), (2,2), (2,3), (2,4), \\ (3,3), (3,4), (4,4)\}$$

$$\text{Dom}(R) = \{1, 2, 3, 4\}, \quad \text{Ran}(R) = \{1, 2, 3, 4\}, \quad \text{공역} = \{1, 2, 3, 4\}$$

모든 원소 a 에 대해 $a \leq a$ 이므로 (a, a) 가 반드시 포함된다 — 이 사실은 5.4절 반사관계로 이어진다.

예제 5-8 — 나누어떨어짐 관계

문제

$A = \{2, 3, 4, 7\}$, $B = \{2, 4, 5, 6\}$ 이고, b 가 a 로 나누어떨어질 때 $a R b$ 로 정의한다. R 과 그 정의역 · 치역 · 공역을 구하라.

증명

b 가 a 의 배수인 순서쌍 (a, b) 를 모은다.

$$R = \{(2,2), (2,4), (2,6), (3,6), (4,4)\}$$

$$\text{Dom}(R) = \{2, 3, 4\}, \quad \text{Ran}(R) = \{2, 4, 6\}, \quad \text{공역} = \{2, 4, 5, 6\}$$

정의역 · 치역과 공역의 차이

공역은 둘째 성분이 뺄 수 있는 후보 전체(집합 B) 이지만, 치역은 실제로 등장한 둘째 성분만 모은다. 여기서 5 와 7 은 어떤 순서쌍에도 나타나지 않으므로 치역에는 없고 공역에만 있다.

예제 5-9 — 멱집합 위의 포함 관계

문제

$A = \{a\}$ 의 멱집합에 대해 $u R v \Leftrightarrow u \subseteq v$ 로 정의할 때, R 과 정의역 · 치역 · 공역을 구하라.

증명

A 의 멱집합은 $\mathcal{P}(A) = \{\emptyset, \{a\}\}$.

부분집합 관계 $\subseteq$ 를 만족하는 순서쌍을 모은다.

$$R = \{(\emptyset, \emptyset), (\emptyset, \{a\}), (\{a\}, \{a\})\}$$

$$\text{Dom}(R) = \{\emptyset, \{a\}\}, \quad \text{Ran}(R) = \{\emptyset, \{a\}\}, \quad \text{공역} = \{\emptyset, \{a\}\}$$

공집합 $\emptyset$ 은 모든 집합의 부분집합이므로 $(\emptyset, \emptyset)$ 과 $(\emptyset, \{a\})$ 가 모두 포함된다. 관계의 원소가 '집합'일 수도 있음을 보여주는 예이다.

n항관계 — 이항관계의 확장

이항관계는 두 집합의 곱집합의 부분집합이었다. 같은 방식으로 세 집합 이상으로 확장할 수 있다.

정의 — n항관계

세 집합 A, B, C 에 대해 곱집합 $A \times B \times C$ 의 부분집합을 삼항관계라 한다. $x \in A, y \in B, z \in C$ 에 대해 $(x, y, z) \in R$ 이면 세 원소가 관계 R 에 있다고 한다. 이 개념은 임의의 n 항관계까지 확장된다.

AI 응용 — n항관계와 데이터베이스, 지식 그래프

관계형 데이터베이스의 테이블 한 행이 바로 n 항관계의 한 순서쌍이다. (학번, 이름, 학과, 학년) 처럼 여러 속성을 묶은 순서 n 중쌍의 집합이 곧 테이블이다.

지식 그래프는 (주어, 술어, 목적어) 삼항관계로 세계 지식을 표현한다. 예: (서울, 수도이다, 대한민국). RAG 시스템은 이 삼항관계 집합에서 질의에 맞는 사실을 찾아 언어모델에 근거로 제공한다.

예제 5-10 — 4항관계

문제

$A = \{C, P\}$, $B = \{5, 6, 7\}$, $C = \{\text{이산수학, 컴파일러}\}$, $D = \{\text{한국, 미국}\}$ 일 때 다음 R 이 $A \times B \times C \times D$ 의 4항관계인지 판단하라.

$R = \{(C, 5, \text{이산수학}, \text{한국}), (C, 6, \text{이산수학}, \text{미국}), (P, 5, \text{컴파일러}, \text{한국}), (P, 7, \text{컴파일러}, \text{한국})\}$

증명

곱집합 $A \times B \times C \times D$ 는 $|A| \cdot |B| \cdot |C| \cdot |D| = 2 \cdot 3 \cdot 2 \cdot 2 = 24$ 개의 순서 4중쌍을 가진다.

R 의 네 순서쌍은 모두 이 24개 중에 들어 있다 (각 성분이 해당 집합의 원소).

따라서 $R \subseteq A \times B \times C \times D$ 이므로 R 은 4항관계이다.

STUDENT TRY

$(C, 8, \text{이산수학}, \text{한국})$ 을 R 에 추가하면 여전히 4항관계인가? 8 이 B 에 없다는 점에 주목하라.

■ 항등관계 — 자기 자신과의 관계

이제부터 관계라 하면 주로 한 집합 위의 이항관계(A 에서 A 로의 관계)를 가리킨다. 그중 가장 단순한 것이 항등관계이다.

정의 — 항등관계

$$I_A = \{ (a, a) : a \in A \}$$

항등관계는 모든 원소를 오직 자기 자신하고만 관계 맺게 한다. 어떤 a 에 대해서도 (a, a) 만 포함하고 서로 다른 두 원소 사이에는 관계가 없다.

예제 5-11

A 가 정수 집합일 때 $R = \{(x, x) : x \text{ 는 정수}\}$ 가 항등관계인지 판단하라.

풀이: 임의의 정수 x 에 대해 (x, x) 가 R 에 속하고, 그 외의 순서쌍은 없다.

정의에 정확히 부합하므로 R 은 항등관계이다.

관계를 표현하는 세 가지 방법

관계는 순서쌍의 집합이지만, 순서쌍을 일일이 나열하는 것은 직관적이지 않다. 그래서 세 가지 표현 방법을 함께 쓴다.

1

화살표 그림

두 집합을 좌우에 두고, 관계 맺는 원소를 화살표로 잇는다. 서로 다른 두 집합 사이 관계에 편리하다.

2

관계행렬

원소를 행과 열로 두고, 관계가 있으면 1, 없으면 0 을 적은 행렬. 계산에 가장 강력하다.

3

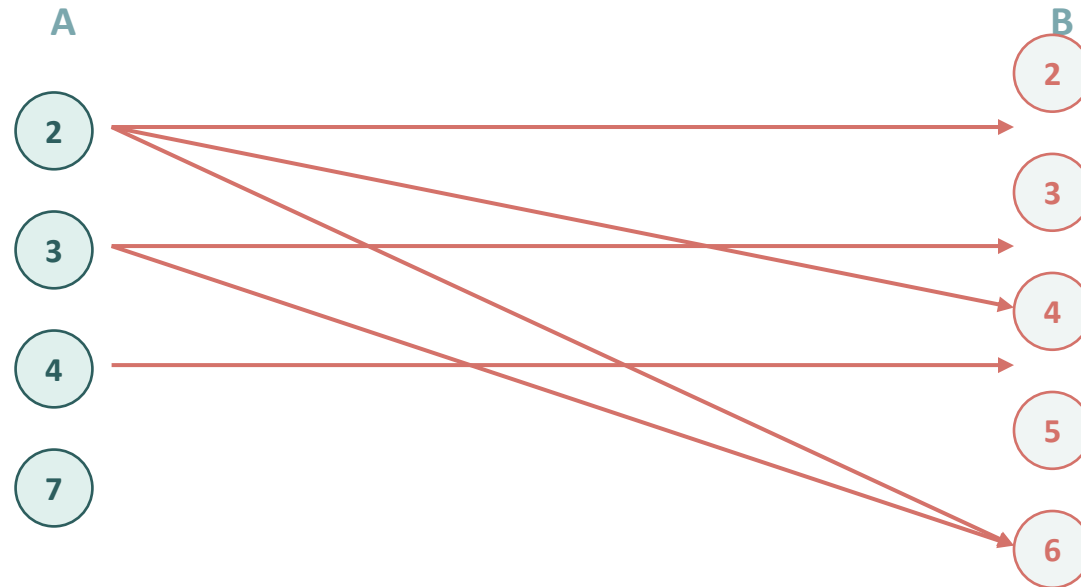
유향 그래프

한 집합 위의 관계에서, 원소를 점으로 두고 관계를 화살표(간선)로 그린다. 경로를 보기에 좋다.

표현 1 — 화살표 그림 (예제 5-12)

문제

$A = \{2, 3, 4, 7\}$ 에서 $B = \{2, 3, 4, 5, 6\}$ 으로의 관계 $R = \{(2, 2), (2, 4), (2, 6), (3, 3), (3, 6), (4, 4)\}$ 를 화살표 그림으로 표현하라.



A 의 원소에서 B 의 원소로 화살표를 그었다. 화살표가 출발하는 원소가 정의역, 도착하는 원소가 치역이다. 7에서 출발하는 화살표가 없으므로 7은 정의역에 없다.

표현 2 — 관계행렬

관계행렬은 관계를 0 과 1 의 행렬로 옮긴 것이다. A 의 원소를 행, B 의 원소를 열에 두고 칸을 채운다

정의 관계행렬

A와 B를 각각 m 개, n 개의 원소를 가진 유한집합이라고 하자. R 을 집합 A에서 집합 B로의 관계라 할 때, 다음 정의에 의해 R 을 $m \times n$ 행렬 $M_R = (m_{ij})$ 로 표기한다.

$$m_{ij} = \begin{cases} 1, & (a_i, b_j) \in R \\ 0, & (a_i, b_j) \notin R \end{cases} \quad \text{단, } \begin{cases} i = 1, 2, \dots, m \\ j = 1, 2, \dots, n \end{cases}$$

이때 행렬 M_R 을 R 의 **관계행렬** relation matrix이라고 한다.

AI 응용 — 관계행렬은 어디에나 있다

추천 시스템의 사용자-상품 행렬, 신경망의 인접행렬 기반 그래프 신경망(GNN), 검색엔진의 문서-단어 행렬이 모두 관계행렬이다. 특히 RAG 시스템은 질의 임베딩과 문서 임베딩 사이의 유사도 행렬을 만들어, 값이 큰 칸(가장 관련 있는 문서)을 골라낸다. 관계행렬의 곱셈은 다음 절에서 '경로'를 세는 핵심 도구가 된다.

예제 5-13 — 관계를 관계행렬로

$A=\{1, 2\}$, $B=\{a, b, c\}$ 이고, 관계 $R = \{(1, a), (1, b), (2, b), (2, c)\}$ 를 관계행렬로 나타내면 다음과 같다.

$$\mathbf{M}_R = \begin{matrix} & \begin{matrix} a & b & c \end{matrix} \\ \begin{matrix} 1 \\ 2 \end{matrix} & \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} \end{matrix}$$

관계행렬 $\mathbf{M}_R$ 의 2행 2열 성분은 1이다. 이는 $(2, b) \in R$ 임을 의미한다. 반면에 2행 1열 성분은 0이고, 이는 $(2, a) \notin R$ 을 의미한다.

예제

$A = \{a_1, a_2, a_3\}$, $B = \{b_1, b_2, b_3, b_4\}$ 일 때, 집합 A 에서 집합 B 로 가는 관계 R 를 관계행렬로 나타내라.

$$R = \{(a_1, b_1), (a_1, b_4), (a_2, b_2), (a_2, b_3), (a_3, b_1), (a_3, b_3)\}$$

관계행렬 $\mathbf{M}_R$ 은 3×4 행렬이고, 이를 나타내면 다음과 같다.

$$\mathbf{M}_R = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

예제 5-15 — 작거나 같음 관계의 행렬

예제 5-7 의 관계 R ($A = \{1,2,3,4\}$ 위의 $a \leq b$) 를 관계행렬로 나타낸다.

	1	2	3	4
1	1	1	1	1
2	0	1	1	1
3	0	0	1	1
4	0	0	0	1

주대각선 위쪽이 모두 1 인 '상삼각' 모양이 나타난다. $a \leq b$ 이려면 행 번호가 열 번호 이하여야 하기 때문이다. 주대각선이 모두 1 인 것은 모든 a 에 대해 $a \leq a$ 임을 뜻한다 (반사관계의 행렬적 특징, 5.4 절).

표현 3 — 유형 그래프

한 집합 위의 관계는 유형 그래프로 그리면 직관적이다. 원소를 점(정점)으로, 관계를 화살표(간선)로 표현한다.

유형 그래프 그리기

집합 A 위의 관계 R 을 그릴 때, A 의 각 원소를 점으로 찍는다. $(a, b) \in R$ 이면 a 에서 b 로 향하는 화살표를 그린다. $(a, a) \in R$ 이면 a 에서 a 로 돌아오는 고리(loop) 를 그린다.

유형 그래프는 화살표를 따라가며 '경로' 를 추적할 수 있어, 다음 절의 경로 · 연결관계 학습에 핵심이 된다.

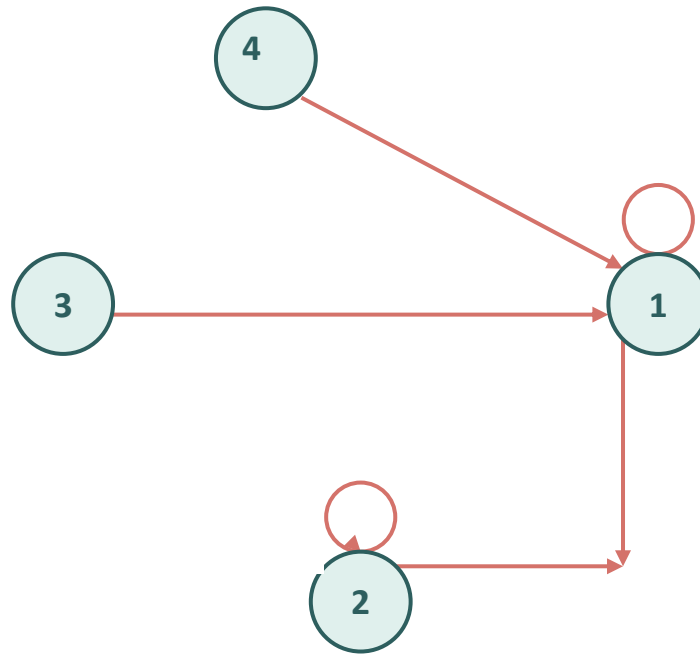
STUDENT TRY

예제 5-7 의 관계 R 을 유형 그래프로 그려 보라. 정점 1, 2, 3, 4 를 찍고, $a \leq b$ 인 모든 (a, b) 에 화살표를 그린다. 고리는 몇 개가 생기는가?

예제 5-16 — 관계를 유향 그래프로

문제

$A = \{1,2,3,4\}$ 위의 관계 $R = \{(1,1), (1,2), (2,1), (2,2), (2,3), (2,4), (3,4), (4,1)\}$ 을 유향 그래프로 그려라



정점 1, 2, 3, 4 를 찍고 각 순서쌍마다 화살표를 그렸다. (1,1) 과 (2,2) 는 고리이다. (1,2) 와 (2,1) 처럼 양방향 화살표가 있으면 두 정점이 서로 오갈 수 있다.

예제 5-17 — 크기 관계의 유형 그래프와 행렬

문제

$X = \{1, 2, 3, 4, 5\}$ 위의 관계 $R = \{(x, y) : x > y\}$ 를 유형 그래프와 관계행렬로 표현하라.

증명

$R = \{(2, 1), (3, 1), (3, 2), (4, 1), (4, 2),$
 $(4, 3), (5, 1), (5, 2), (5, 3), (5, 4)\}$

	1	2	3	4	5
1	0	0	0	0	0
2	1	0	0	0	0
3	1	1	0	0	0
4	1	1	1	0	0
5	1	1	1	1	0

$x > y$ 이려면 행 번호가 열 번호보다 커야 하므로, 관계행렬은 주대각선 아래쪽만 1 인 '하삼각' 모양이다. 주대각선이 모두 0 인 것은 어떤 x 도 $x > x$ 일 수 없음을 뜻한다 (비반사관계, 5.4절).

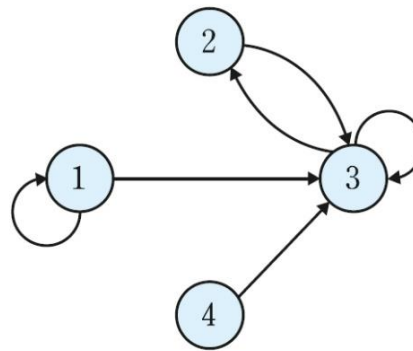
예제 5-18 — 유향 그래프에서 관계 읽기

문제 어떤 유향 그래프에서 정점 1 은 1·3 과, 2 는 3 과, 3 은 2·3 과, 4 는 3 과 간선으로 연결되어 있다. 관계 R 을 구하라.

증명

유향 그래프의 간선 하나하나가 관계의 순서쌍 하나이다.
각 정점에서 출발하는 간선을 (출발, 도착) 순서쌍으로 옮긴다.

$R = \{(1,1), (1,3), (2,3), (3,2), (3,3), (4,3)\}$



유향 그래프에서 관계로 옮길 때는 '간선의 방향' 을 그대로 순서쌍의 순서로 적어야 한다. 방향을 무시하면 다른 관계가 된다.

Lean 4 — 관계를 어떻게 표현하는가

Lean 4 에서 한 집합 위의 관계는 '두 인수를 받아 참 · 거짓을 돌려주는 함수' 로 표현한다. 타입 Prop 은 '명제(참이거나 거짓)' 를 뜻한다.

```
-- 관계: 두 인수를 받아 명제를 돌려주는 함수
def Le : Nat → Nat → Prop := fun a b => a ≤ b

-- a R b 는 Lean 에서 R a b 로 쓴다.
example : Le 1 3 := by
  show 1 ≤ 3      -- Le 1 3 의 정의를 펼친 모습
  omega          -- 자연수 부등식이므로 omega 로 닫음

-- 1 과 3 은 관계에 있다: (1, 3) ∈ Le
```

lean

수학의 ' $a R b$ ' 가 Lean 의 ' $R a b$ ' 와 정확히 대응한다. show 는 현재 목표를 정의를 펼친 형태로 다시 적어 가독성을 높인다.

수학 ↔ Lean — 항등관계

수학 진술

증명

항등관계의 정의:

$$I_A = \{(x, x) : x \in A\}$$

곧, $x R y$ 가 성립할
필요충분조건은 $x = y$.

확인할 성질:

모든 x 에 대해 $x I_A x$.

증명: $x = x$ 는 항상 참
이므로 (자기 자신과는
언제나 같으므로) 성립.

Lean 4 정의와 증명

lean

```
-- 항등관계: x 와 y 가 같을 때만 성립
def IdRel (A : Type) :
  A → A → Prop :=
  fun x y => x = y

-- 모든 x 에 대해 x 는 자기
-- 자신과 항등관계에 있다
example (x : Nat) :
  IdRel Nat x x := by
  show x = x
  rfl      -- 자기 자신과 같음
```

수학의 정의 ' $I_A = \{(x,x)\}$ ' 가 Lean 의 ' $\text{fun } x \ y \Rightarrow x = y$ ' 와 한 줄로 대응한다. `rfl` 은 ' $x = x$ ' 를 닫는다.

Python — 관계행렬과 유향 그래프

관계를 순서쌍 집합으로 두면, 관계행렬과 유향 그래프를 코드로 자동 생성할 수 있다.

python

```
A = [1, 2, 3, 4]
R = {(1,1), (1,2), (2,1), (2,2), (2,3), (2,4), (3,4), (4,1)}

# 관계행렬: 관계에 있으면 1, 없으면 0
idx = {v: i for i, v in enumerate(A)}
M = [[0]*len(A) for _ in A]
for (a, b) in R:
    M[idx[a]][idx[b]] = 1
for row in M:
    print(row)

# 유향 그래프: networkx 의 방향 그래프
import networkx as nx
G = nx.DiGraph(); G.add_nodes_from(A); G.add_edges_from(R)
```

관계행렬 M 과 그래프 G 는 같은 관계 R 의 서로 다른 표현이다. `networkx` 는 경로 탐색 함수를 그대로 제공한다.

PART

5.3

경로

유향 그래프 위의 경로와 순환, 연결관계,
그리고 관계행렬의 부울곱으로 경로를 세는 방법을 다룬다.

경로 — 화살표를 따라가기

유형 그래프에서 한 정점을 출발해 화살표를 차례로 따라 다른 정점에 이르는 길을 경로(path)라 한다. 지나간 화살표(간선)의 개수를 경로의 길이라 한다.

정의 경로

R 을 집합 A 에 관한 관계라고 하자. $a, b \in A$ 에 대해 a 에서 b 까지의 길이가 n 인 경로 path of length n 는 a 에서 시작하여 b 로 끝나는 유한수열 $\pi = a, x_1, x_2, \dots, x_{n-1}, b$ 를 말한다. 각 원소들은 $aRx_1, x_1Rx_2, \dots, x_{n-1}Rb$ 를 만족해야 한다.

AI 연결 — 멀티홉 추론

지식 그래프에서 'A의 지도교수의 출신 대학'을 묻는 질문은 두 칸을 따라가는 길이 2 경로 탐색이다. 최신 RAG 시스템의 멀티홉(multi-hop) 추론이 바로 이 경로 따라가기이다. 한 번의 검색으로 답이 안 나오면, 중간 결과를 발판 삼아 다음 칸으로 이동한다.

순환 — 제자리로 돌아오는 경로

출발한 정점으로 다시 돌아오는 경로를 순환(cycle)이라 한다. 시작점과 끝점이 같은 경로이다.

정의 — 순환

경로 $v_0, v_1, \dots, v_k$ 에서 $v_0 = v_k$ 이면 이 경로를 순환이라 한다. 특히 $(a, a) \in R$ 이면 a 에서 a 로 가는 길이 1 인 순환이며, 유한 그래프에서는 고리(loop) 로 나타난다.

예: 경로 1, 2, 5, 1 은 길이 3 인 순환이고, 2, 2 는 길이 1 인 순환이다. 여러 경로의 종류는 8장 그래프 이론에서 더 자세히 다룬다.

용어 정리

경로는 '어딘가에서 어딘가로' 가는 길, 순환은 '제자리로 돌아오는' 특별한 경로이다. 모든 순환은 경로이지만, 모든 경로가 순환은 아니다.

STUDENT TRY

여러분이 아는 일상의 '순환' 을 하나 떠올려 보라. 출퇴근 경로는 순환인가, 단순 경로인가?

정의 5-8 관계와 경로

n 은 양의 정수이다.

- (1) 집합 A 에 관한 관계 R 에 대해 길이가 n 인 관계 R^n 을 정의하면, $xR^n y$ 는 x 에서 시작하여 y 로 끝나는 길이가 n 인 경로가 있음을 의미한다.
- (2) 집합 A 에 관한 관계 R 에 대해 연결관계(connectivity relation) R^∞ 를 정의하면, $xR^\infty y$ 는 x 에서 시작하여 y 로 끝나는 어떤 경로가 있음을 의미한다. 즉 xRy 혹은 xR^2y 혹은 xR^3y 혹은 ...이다.
- (3) 집합 A 에 관한 관계 R 에 대해 도달관계(reachability relation) R^* 를 정의하면, $x = y$ 혹은 연결관계인 $xR^\infty y$ 이다. 이를 관계행렬과 연결하면 도달관계는 관계행렬의 단위행렬(I) 혹은 M_{R^∞} 이다. 또한 유한 그래프에서 임의의 두 정점 사이에 도달관계가 성립하면 강하게 연결(strongly connected)되었다고 한다.

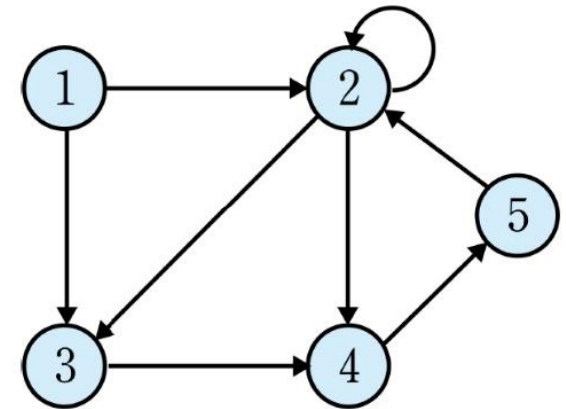
예제 5-19 — 길이 2 경로 모두 찾기

문제

$A = \{1, 2, 3, 4, 5\}$ 이고 관계 $R = \{(1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 5), (5, 2)\}$ 일 때, R 로부터 길이가 2 인 경로를 모두 찾으라.

$1 R^2 2$ ($\because 1 R 2$ 이고 $2 R 2$),
 $1 R^2 4$ ($\because 1 R 2$ 이고 $2 R 4$),
 $2 R^2 3$ ($\because 2 R 2$ 이고 $2 R 3$),
 $2 R^2 5$ ($\because 2 R 4$ 이고 $4 R 5$),
 $4 R^2 2$ ($\because 4 R 5$ 이고 $5 R 2$)
 $5 R^2 3$ ($\because 5 R 2$ 이고 $2 R 3$)

$1 R^2 3$ ($\because 1 R 2$ 이고 $2 R 3$),
 $2 R^2 2$ ($\because 2 R 2$ 이고 $2 R 2$),
 $2 R^2 4$ ($\because 2 R 2$ 이고 $2 R 4$),
 $3 R^2 5$ ($\because 3 R 4$ 이고 $4 R 5$),
 $5 R^2 2$ ($\because 5 R 2$ 이고 $2 R 2$),
 $5 R^2 4$ ($\because 5 R 2$ 이고 $2 R 4$),



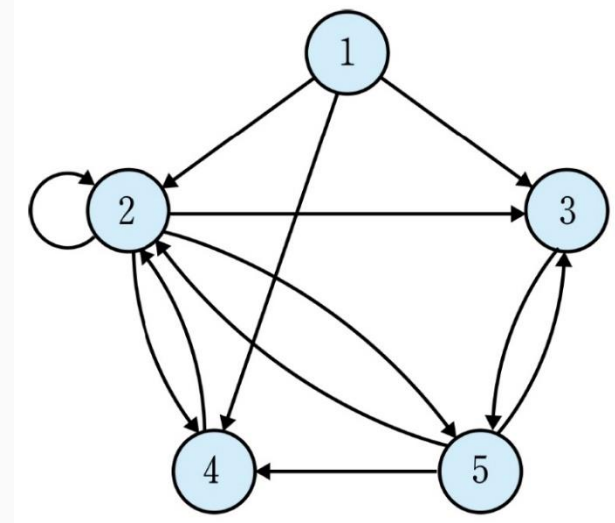
길이 2 경로 $a \rightarrow b \rightarrow c$ 는 $(a, b) \in R$ 이고 $(b, c) \in R$ 인 경우이다. 모든 정점에서 출발해 두 칸 따라간 결과를 (a, c) 로 적는다.

예제 5-19 — 풀이 (길이 2 경로 열거)

증명

그러므로 $R^2 = \{(1, 2), (1, 3), (1, 4), (2, 2), (2, 3), (2, 4), (2, 5), (3, 5), (4, 2), (5, 2), (5, 3), (5, 4)\}$ 이다.

R^2 의 유형 그래프를 그리면 다음과 같다.



길이 n 경로 — 관계의 거듭제곱

길이 2 경로를 모은 것이 R^2 이듯, 길이 n 경로를 모은 것이 R^n 이다. 한 칸씩 더 따라가며 정의한다.

정의 — 관계의 거듭제곱

$$R^n = R \circ R^{n-1}, \quad R^1 = R$$

R^1 은 R 자체이고, R^n 은 'R 을 한 번 한 다음, 남은 n-1 칸을 R 로 가는' 합성이다. $(a, c) \in R^n$ 은 'a 에서 c 로 가는 길이 n 경로가 있다' 를 뜻한다.

R^n 은 행렬 곱과 닮았다

관계행렬로 보면 R^n 의 관계행렬은 M_R 을 n 번 부울 곱한 것이다. 일반 행렬 곱이 '경로의 개수' 를 센다면, 부울 곱은 '경로의 존재 여부(있다 1 / 없다 0)' 만 따진다. 다음 슬라이드에서 부울곱을 정확히 정의한다.

연결관계 — 길이를 따지지 않는 도달 가능성

경로의 길이를 따지지 않고, '어떤 길이로든 닿을 수 있는가' 만 묻는 관계를 연결관계라 한다.

정의 — 연결관계

$$R^{\infty} = \bigcup_{n=1}^{\infty} R^n$$

$x R^{\infty} y$ 는 'x 에서 y 로 가는 경로가 적어도 하나 있다 (길이 무관)' 를 뜻한다.
. R^{∞} 는 $R^1, R^2, R^3, \dots$ 을 모두 합한 것이다.

AI 응용 — 도달 가능성과 추천

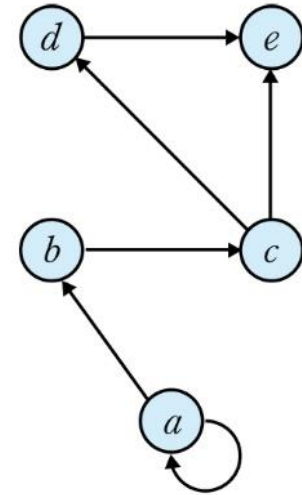
소셜 네트워크에서 '나와 어떻게든 연결된 모든 사람' 이 곧 연결관계 R^{∞} 이다. 추천 시스템은 '친구의 친구의 친구...' 를 따라가며 닿을 수 있는 사용자를 후보로 삼는다. 지식 그래프 기반 RAG 에서도 질의 개체에서 도달 가능한 사실의 범위를 R^{∞} 가 결정한다.

예제

집합 $A = \{a, b, c, d, e\}$ 에 관한 관계 $R = \{(a, a), (a, b), (b, c), (c, d), (c, e), (d, e)\}$ 에 대해 다음을 구하라.

(a) R^2

(b) 연결관계 R^∞



R^∞ 를 구하려면 첫 번째 정점에서 두 번째 정점까지 경로의 길이와는 상관없이 경로가 있는 모든 순서쌍을 구하면 된다. 즉 R^∞ 는 다음과 같다.

핵심 아이디어 — 관계의 거듭제곱

R 은 한 칸, R^2 은 두 칸, R^3 은 세 칸 떨어진 관계이다. ' k 칸 떨어진 모든 쌍'을 알면, 그래프에서 누가 누구에게 닿을 수 있는지 거리별로 파악된다.

풀이

(a) R^2 을 나열하면 다음과 같다.

aRa^0 이고 aRa^0 이므로 aR^2a 이다. aRa^0 이고 aRb^0 이므로 aR^2b 이다.

aRb^0 이고 bRc^0 이므로 aR^2c 이다. bRc^0 이고 cRd^0 이므로 bR^2d 이다.

bRc^0 이고 cRe^0 이므로 bR^2e 이다. cRd^0 이고 dRe^0 이므로 cR^2e 이다.

그러므로 $R^2 = \{(a, a), (a, b), (a, c), (b, d), (b, e), (c, e)\}$

각 정점에서 화살표를 끝까지 따라가며 닿는 모든 정점을 모았다. a 는 자기 고리 때문에 (a,a) 도 포함한다

R^∞ 는 길이를 따지지 않고 '경로가 있는 모든 (시작, 끝)'을 모은다.

a 에서: a, b, c, d, e 모두 도달 가능 ($a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$)

b 에서: c, d, e 도달

c 에서: d, e 도달

d 에서: e 도달

$R^\infty = \{(a,a), (a,b), (a,c), (a,d), (a,e), (b,c), (b,d), (b,e), (c,d), (c,e), (d,e)\}$

관계행렬의 부울곱

부울곱은 일반 행렬 곱에서 덧셈을 OR($\vee$), 곱셈을 AND($\wedge$) 로 바꾼 연산이다. 결과는 0 또는 1 뿐이다.

정의 — 부울곱

$$(M_R \odot M_S)_{ij} = \bigvee_k ((M_R)_{ik} \wedge (M_S)_{kj})$$

i행 j열의 성분은 '중간 정점 k 를 거쳐 i 에서 j 로 닿는 길이 2 경로가 하나라도 있는가' 를 OR 로 묻는다. (i,k) 도 있고 (k,j) 도 있으면 그 k 를 거쳐 닿는다.

길이 2 경로의 행렬식

$$M_{R^2} = M_R \odot M_R$$

곱, R^2 의 관계행렬은 M_R 을 자기 자신과 부울 곱한 것이다.

정리 5-2 부울곱을 이용해 길이가 2인 경로 찾기

집합 $A = \{a_1, a_2, \dots, a_n\}$ 에 관한 관계 R 의 관계행렬 M_R 을 $n \times n$ 행렬 $M_R = (m_{ij})$ 라고 하자. 이때 길이가 2인 경로는 $M_{R^2} = M_R \odot M_R$ 이다. 여기서 M_{R^2} 은 $(M_R)^2$ 으로도 표기한다.

증명

(증명) $M(R^2)$ 의 i 행 j 열 성분이 1인 경우를 따져 본다.

- (1) 부울곱의 정의에 의해 $(M_R \odot M_R)(i,j) = 1$ 이려면, 어떤 중간 k 에 대해 $(M_R)(i,k) = 1$ 이고 $(M_R)(k,j) = 1$ 이어야 한다.
- (2) $(M_R)(i,k) = 1$ 은 $(i, k) \in R$, $(M_R)(k,j) = 1$ 은 $(k, j) \in R$ 을 뜻한다.
- (3) 곧 'i에서 k로, 다시 k에서 j로 가는 길이 2 경로가 존재' 한다는 것이며, 이는 정확히 $(i, j) \in R^2$ 의 정의이다.
- (4) 따라서 $(M_R \odot M_R)(i,j) = 1 \iff (i,j) \in R^2$. 곧 $M(R^2) = M_R \odot M_R$. ■

예제 5-21 — 부울곱으로 R^2 구하기

예제 5-20의 관계 R 을 관계행렬로 두고, 부울곱 $M_R \odot M_R$ 로 R^2 를 구한다. 행·열 순서는 a, b, c, d, e 이다.

$$M_R = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

	M_R				
	a	b	c	d	e
a	1	1	0	0	0
b	0	0	1	0	0
c	0	0	0	1	1
d	0	0	0	0	1
e	0	0	0	0	0

	$M_R \odot M_R$				
	a	b	c	d	e
a	1	1	1	0	0
b	0	0	0	1	1
c	0	0	0	0	1
d	0	0	0	0	0
e	0	0	0	0	0

부울곱 결과를 순서쌍으로 읽으면 $R^2 = \{(a,a), (a,b), (a,c), (b,d), (b,e), (c,e)\}$ 이다. 예제 5-20에서 손으로 찾은 R^2 와 정확히 일치한다 — 정리 5-2가 옳음을 확인한다.

정리 5-3 부울곱을 이용해 길이가 n 인 경로 찾기

$n \geq 1$ 이고 R 은 유한집합 A 에 관한 관계일 때, 길이가 n 인 경로는 다음과 같다.

$$M_{R^n} = \underbrace{M_R \odot M_R \odot M_R \odot \cdots \odot M_R}_{n\text{개}}$$

즉 n 개의 M_R 을 부울곱한 것이다.

경로의 합성과 역경로

정의 — 경로의 합성

경로 π_1 의 마지막 정점과 경로 π_2 의 첫 정점이 같으면, π_1 을 간 뒤 이어서 π_2 를 가는 긴 경로를 만들 수 있다. 이를 합성 경로 $\pi_1 \cdot \pi_2$ 라 한다. 예제 5-22: $\pi_1 = 1,2,3$ 과 $\pi_2 = 3,5,6,2,4$ 의 합성은 $\pi_1 \cdot \pi_2 = 1,2,3,5,6,2,4$ 이다.

정의 — 역경로

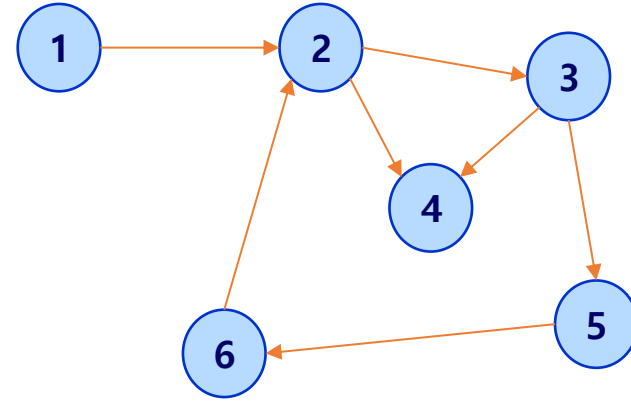
경로 $\pi = v_0, v_1, \dots, v_k$ 의 정점 순서를 거꾸로 뒤집은 $v_k, \dots, v_1, v_0$ 을 역경로 π^{-1} 라 한다. 예제 5-23: $\pi_1 = 1,2,3$ 의 역경로는 $3,2,1$ 이고, $\pi_2 = 3,5,6,2,4$ 의 역경로는 $4,2,6,5,3$ 이다.

역경로가 실제 경로가 되려면

정점 순서를 뒤집는 것만으로 역경로의 '나열' 은 얻지만, 그것이 그래프에서 실제로 다닐 수 있는 경로하려면 거꾸로 향한 간선들이 모두 관계 R 에 있어야 한다. 무향 그래프에서는 늘 가능하지만, 유향 그래프에서는 보장되지 않는다.

예제

관계 R에 대한 유향 그래프가 다음과 같을 때 두 경로 $\pi_1 = 1, 2, 3$ 과 $\pi_2 = 3, 5, 6, 2, 4$ 에 대해 합성 $\pi_1 \cdot \pi_2$ 을 구하라.



풀이

π_1 의 마지막 정점 3과 π_2 의 시작 정점이 3이 같으므로 합성은 가능하다. 두 경로 π_1 과 π_2 의 합성은 $\pi_1 \cdot \pi_2 = 1, 2, 3, 5, 6, 2, 4$ 이다.

Lean 4 — 관계의 합성 정의

관계 R 을 따라간 뒤 S 를 따라가는 합성 관계를 Lean 으로 정의한다. '중간 정점 b 가 존재한다' 를 $\exists$ 로 표현한다.

```
-- 합성: a 에서 b 로 R, b 에서 c 로 S 면, a 에서 c 로 (S∘R)
def Comp {A : Type} (R S : A → A → Prop) :
  A → A → Prop :=
  fun a c => ∃ b, R a b ∧ S b c

-- R 의 제곱: R 을 두 번 합성
def RPow2 {A : Type} (R : A → A → Prop) :
  A → A → Prop :=
  Comp R R      -- 길이 2 경로의 관계
```

lean

수학의 ' $S \circ R = \{(a,c) : \exists b, (a,b) \in R \text{ 이고 } (b,c) \in S\}$ ' 가 Lean 의 ' $\text{fun } a \text{ } c \Rightarrow \exists b, R \text{ } a \text{ } b \wedge S \text{ } b \text{ } c$ ' 와 한 줄로 대응한다.

수학 $\leftrightarrow$ Lean — R^2 에 순서쌍이 들어감

수학 추론

증명

예제 5-20 의 R 에서
 $(a, c) \in R^2$ 임을 보인다.

R^2 의 정의:

$(a, c) \in R^2$
 $\Leftrightarrow \exists m, (a, m) \in R$
그리고 $(m, c) \in R$.

중간 정점으로 b 를 잡으면
 $(a, b) \in R$ 이고
 $(b, c) \in R$ 이다.

따라서 그러한 m 이
존재하므로 $(a, c) \in R^2$.

Lean 4 증명

lean

```
-- R: a 에서 b, b 에서 c 로
-- 두 사실로부터 a R^2 c
example {A : Type}
  (R : A → A → Prop)
  (a b c : A)
  (hab : R a b)
  (hbc : R b c) :
  Comp R R a c := by
  -- 목표:  $\exists m, R a m \wedge R m c$ 
  -- 중간 정점으로 b 를 제시
  exact ⟨b, hab, hbc⟩
```

수학의 '중간 정점 b 를 잡으면' 이 Lean 의 ' $\langle b, hab, hbc \rangle$ ' 와 대응한다. 존재($\exists$) 증명은 '증인을 제시' 하는 것이다.

Lean 4 — 합성의 결합법칙

경로를 이어 붙이는 순서를 바꿔 묶어도 결과가 같다. 곧 $(T \circ S) \circ R = T \circ (S \circ R)$ 이다. 이를 Lean 으로 증명한다.

```
example {A : Type} (R S T : A → A → Prop) (a d : A) :  
  Comp (Comp R S) T a d ↔ Comp R (Comp S T) a d := by  
  constructor  
  · rintro ⟨c, ⟨b, hRab, hSbc⟩, hTcd⟩  
    -- 왼쪽 묶음을 풀면 b, c 라는 두 중간 정점이 나온다  
    exact ⟨b, hRab, c, hSbc, hTcd⟩  
  · rintro ⟨b, hRab, c, hSbc, hTcd⟩  
    -- 오른쪽 묶음을 풀어 같은 b, c 로 왼쪽을 구성  
    exact ⟨c, ⟨b, hRab, hSbc⟩, hTcd⟩
```

lean

rintro 는 가정을 받으면서 동시에 분해한다. 결합법칙의 증명은 결국 '같은 중간 정점들을 다르게 묶었을 뿐' 임을 보인다.

Python — 부울곱으로 R^n 계산

관계행렬의 부울곱을 코드로 구현하면, 임의의 길이 n 경로 관계 R^n 를 자동으로 계산할 수 있다.

```
def bool_mul(A, B):  
    n = len(A)  
    C = [[0]*n for _ in range(n)]  
    for i in range(n):  
        for j in range(n):  
            # OR 로 합치고 AND 로 곱한다  
            C[i][j] = 1 if any(A[i][k] and B[k][j]  
                               for k in range(n)) else 0  
    return C  
  
M = [[1,1,0,0,0],[0,0,1,0,0],[0,0,0,1,1],  
      [0,0,0,0,1],[0,0,0,0,0]] # 예제 5-20 의 M_R  
M2 = bool_mul(M, M)             #  $R^2$  의 관계행렬  
M3 = bool_mul(M2, M)           #  $R^3$  의 관계행렬
```

python

`any(...)` 가 OR($\vee$), `and` 가 AND($\wedge$) 역할을 한다. M2 를 출력하면 예제 5-21 의 결과와 같다.

5.3 요약 — 경로

경로 · 순환

화살표를 따라가는 길. 제자리로 오면 순환.

관계의 거듭제곱

R^n = 길이 n 경로의 관계. $R^2 = R \circ R$.

연결관계 R_∞

길이 무관, 닿을 수 있는 모든 쌍의 모임.

부울곱 (정리 5-2)

$M(R^2) = M_R \odot M_R$. OR·AND 로 경로 존재를 셈.

Lean 대응

$\text{Comp } R \ S = \text{fun } a \ c \Rightarrow \exists b, R \ a \ b \wedge S \ b \ c$.

PART

5.4

관계의 성질

반사 · 대칭 · 추이 등 관계가 가질 수 있는 성질을 분류하고,
동치관계와 동치류의 개념을 익힌다.

관계의 성질 — 관계를 분류하는 잣대

모든 관계가 똑같이 생긴 것은 아니다. '자기 자신과 관계 맺는가', '관계가 양방향인가', '징검다리로 이어지는가' 같은 잣대로 관계를 분류한다. 이 잣대들이 관계의 성질이다.

반사성

모든 원소가 자기 자신과 관계를 맺는가

대칭성

$a R b$ 이면 $b R a$ 도 성립하는가

반대칭성

양방향 관계는 같은 원소뿐인가

추이성

징검다리로 이르면 직접 이어지는가

이 성질들을 조합하면 동치관계와 부분순서관계(7장) 같은 특별히 중요한 관계가 정의된다.

반사관계 — 자기 자신과의 관계

집합 A 의 모든 원소가 자기 자신과 관계를 맺을 때, 그 관계를 반사관계라 한다.

정의 — 반사관계

$$\forall a \in A, \quad (a, a) \in R$$

'모든 a 에 대해 $(a, a) \in R$ '. 단 한 원소라도 자기 자신과 관계가 없으면 반사관계가 아니다.

유향 그래프 · 관계행렬에서의 특징

유향 그래프: 모든 정점에 고리(loop)가 있다. 관계행렬: 주대각선 성분이 모두 1 이다.

일상 비유

' a 는 a 와 같은 학교다', ' a 는 a 와 키가 같다' 는 반사관계이다. 누구나 자기 자신과는 같은 학교, 같은 키이기 때문이다. 반면 ' a 는 a 보다 크다' 는 반사관계가 아니다.

예제 5-24 — 반사관계 판별

문제

$A = \{1, 2, 3\}$ 위의 다음 두 관계가 반사관계인지 판별하라.

$R1 = \{(1,1), (1,2), (2,2), (3,3)\}$ $R2 = \{(1,1), (2,2), (2,3)\}$

증명

반사관계이려면 $(1,1), (2,2), (3,3)$ 이 모두 들어 있어야 한다.

$R1: (1,1) \circ, (2,2) \circ, (3,3) \circ \Rightarrow$ 세 쌍이 모두 있으므로 반사관계이다.

$R2: (1,1) \circ, (2,2) \circ, (3,3) \times \Rightarrow (3,3)$ 이 없으므로 반사관계가 아니다.

반사성은 '예외를 허용하지 않는' 성질이다. 한 원소라도 빠지면 곧바로 탈락한다.

STUDENT TRY

$R2$ 를 반사관계로 만들려면 어떤 순서쌍을 추가해야 하는가? 최소한 몇 개가 필요한가?

비반사관계 — 자기 자신과 결코 관계 없음

반사관계와 정반대로, 어떤 원소도 자기 자신과 관계를 맺지 않을 때 비반사관계라 한다.

정의 — 비반사관계

$$\forall a \in A, \quad (a, a) \notin R$$

반사도 비반사도 아닌 관계가 있다

반사관계는 '모든 (a,a) 가 있음', 비반사관계는 '모든 (a,a) 가 없음' 이다. 둘은 서로의 부정이 아니다. 어떤 (a,a) 는 있고 어떤 (a,a) 는 없는 관계는 반사관계도 비반사관계도 아니다. '반사가 아니다' 가 곧 '비반사다' 를 뜻하지 않음에 주의한다.

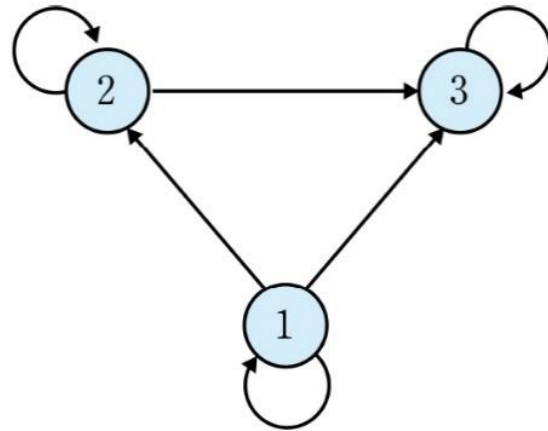
비반사관계의 특징

유형 그래프: 고리가 하나도 없다. 관계행렬: 주대각선 성분이 모두 0 이다. 예: $a < b$, $a \neq b$.

예제

- (a) $A = \{1, 2, 3\}$, $R = \{(a, b) \in A \times A \mid a \leq b\}$ 일 때, R 이 반사관계임을 보여라.
- (b) $A = \{1, 2, 3\}$, $R = \{(a, b) \in A \times A \mid a \neq b\}$ 일 때, R 이 비반사관계임을 보여라.
- (c) $A = \{1, 2, 3\}$, $R = \{(1, 1), (1, 2), (2, 2), (3, 2)\}$ 일 때, R 이 반사관계인지, 비반사관계인지 판단하라.

풀이

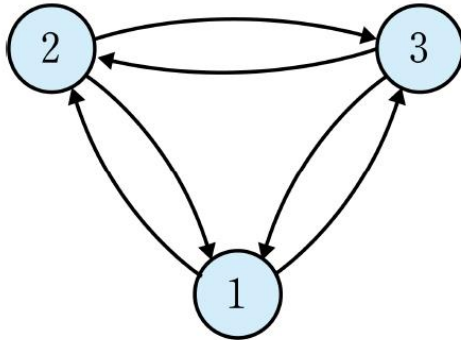


$$M_R = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

반사관계의 정의에 의해 모든 $a \in A$ 에 대해서 순서쌍 $(a, a) \in R$ 인지 확인하면 된다. $A = \{1, 2, 3\}$ 이므로 R 안에 순서쌍 $(1, 1), (2, 2), (3, 3)$ 이 모두 있으므로 R 은 반사관계이다.

(b) R을 구하면 $R = \{(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)\}$ 이다.

$A = \{1, 2, 3\}$ 이므로 이를 유향 그래프와 관계행렬로 표현하면 다음과 같다.



$$M_R = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

비반사관계의 정의에 의해 모든 $a \in A$ 에 대해 순서쌍 $(a, a) \notin R$ 인지 확인하면 된다. $A = \{1, 2, 3\}$ 이고, R안에 순서쌍 $(1, 1), (2, 2), (3, 3)$ 이 모두 없으므로 R은 비반사관계이다.

(c) R을 구하면 $R = \{(1, 1), (1, 2), (2, 2), (3, 2)\}$ 이다. 순서쌍 $(1, 1) \in R$ 이므로 R은 비반사관계가 아니다.

예제 5-25 — 비반사관계 판별

문제

$A = \{1, 2, 3\}$ 위의 다음 관계가 비반사관계인지 판별하라.

$R3 = \{(1,2), (2,1), (2,3)\}$ $R4 = \{(1,1), (1,2), (2,3)\}$

증명

비반사관계하려면 $(1,1), (2,2), (3,3)$ 이 하나도 없어야 한다.

$R3$: 주대각선 쌍이 하나도 없다 $\Rightarrow$ 비반사관계이다.

$R4$: $(1,1)$ 이 들어 있다 $\Rightarrow$ 비반사관계가 아니다.

$R4$ 는 비반사관계도 아니지만, $(2,2) \cdot (3,3)$ 이 없으므로 반사관계도 아니다.

한 관계가 반사 · 비반사 어느 쪽에도 속하지 않을 수 있음을 $R4$ 가 잘 보여 준다.

대칭관계 — 양방향 관계

a 가 b 와 관계를 맺으면 b 도 반드시 a 와 관계를 맺을 때, 그 관계를 대칭관계라 한다.

정의 — 대칭관계

$$(a, b) \in R \implies (b, a) \in R$$

유향 그래프: 화살표가 항상 양방향으로 짝지어 나타난다. 관계행렬: 주대각선을 기준으로 대칭이다 ($m(i,j) = m(j,i)$).

일상 비유와 AI 연결

'a 는 b 의 친구다' 는 대칭관계이다 — 내가 누군가의 친구면 그 사람도 내 친구이다. 페이스북의 친구 그래프가 대칭(무향), X(트위터)의 팔로우 그래프는 비대칭(유향)이다. 추천 시스템은 대칭 관계에서는 무향 그래프 알고리즘을, 비대칭 관계에서는 유향 그래프 알고리즘을 쓴다.

예제 5-26 — 대칭관계 판별

문제

$A = \{1, 2, 3\}$ 위의 다음 관계가 대칭관계인지 판별하라.

$R5 = \{(1,2), (2,1), (2,3), (3,2)\}$ $R6 = \{(1,2), (2,1), (2,3)\}$

증명

대칭관계이라면 (a,b) 가 있을 때마다 (b,a) 도 있어야 한다.

$R5: (1,2) \leftrightarrow (2,1) \bigcirc, (2,3) \leftrightarrow (3,2) \bigcirc \Rightarrow$ 모든 쌍이 짝을 이루므로 대칭관계이다.

$R6: (2,3)$ 은 있지만 $(3,2)$ 가 없다 $\Rightarrow$ 대칭관계가 아니다.

대칭성은 '짝꿍 확인' 이다. 순서를 뒤집은 쌍이 빠짐없이 들어 있는지 확인한다.

STUDENT TRY

$R6$ 을 대칭관계로 만들려면 무엇을 추가해야 하는가? 항등관계는 대칭관계인가? 직접 확인해 보라.

비대칭관계 — 결코 양방향이 아님

$a R b$ 이면 $b R a$ 는 결코 성립하지 않을 때, 그 관계를 비대칭관계라 한다.

정의 — 비대칭관계

$$(a, b) \in R \implies (b, a) \notin R$$

비대칭관계는 (a,b) 와 (b,a) 가 동시에 있을 수 없다. 특히 (a,a) 도 있을 수 없으므로 비대칭관계는 반드시 비반사관계이다.

예제 5-27

$A = \{1,2,3\}$ 위의 $R = \{(1,2), (2,3), (1,3)\}$ 은 비대칭관계인가? $(2,1), (3,2), (3,1)$ 이 모두 없으므로 비대칭관계이다.

비대칭 vs 다음에 배울 반대칭

비대칭은 ' (a,a) 조차 안 됨', 반대칭은 ' (a,a) 는 괜찮음'. 이 미세한 차이를 다음 슬라이드에서 본다.

반대칭관계 — 양방향이면 같은 원소

$a R b$ 와 $b R a$ 가 동시에 성립하는 경우는 오직 a 와 b 가 같을 때뿐일 때, 반대칭관계라 한다.

정의 — 반대칭관계

$$(a, b) \in R \wedge (b, a) \in R \implies a = b$$

반대칭관계는 (a, a) 형태의 쌍은 허용한다. 다만 서로 다른 두 원소 사이에 양방향 관계가 있으면 안 된다.

예제 5-28

$a \leq b$ 관계는 반대칭관계인가? $a \leq b$ 이고 $b \leq a$ 이면 반드시 $a = b$ 이다. 따라서 $\leq$ 는 반대칭관계이다 — 이는 7장 부분순서관계의 핵심 성질이다.

왜 중요한가

반대칭성은 '순서' 를 만든다. $a \leq b$ 이고 $b \leq a$ 인데 $a \neq b$ 라면 둘의 순위를 정할 수 없다. 반대칭성이 있어야 모든 원소를 일렬로 줄 세우는 것이 가능해진다.

대칭성 · 반대칭성 — 관계행렬로 보기

관계행렬을 보면 대칭성과 반대칭성을 한눈에 판별할 수 있다.

	1	2	3
1	0	1	0
2	1	0	1
3	0	1	0

대칭관계

	1	2	3
1	1	1	1
2	0	1	1
3	0	0	1

반대칭관계

대칭관계: 주대각선을 접는 선으로 삼아 행렬이 거울처럼 일치한다 ($m(i,j) = m(j,i)$).

반대칭관계: 서로 다른 i, j 에 대해 $m(i,j)$ 와 $m(j,i)$ 가 동시에 1인 경우가 없다 (주대각선은 무관).

관계행렬을 받았을 때, 주대각선 위쪽과 아래쪽을 비교하면 두 성질을 빠르게 가려낼 수 있다.

대칭 · 반대칭 — 흔한 오개념 정리

오개념 1 — 반대칭은 대칭의 반대가 아니다

'대칭이 아니다' 와 '반대칭이다' 는 다른 말이다. 한 관계가 대칭이면서 동시에 반대칭일 수도 있고 (예: 항등관계), 대칭도 반대칭도 아닐 수도 있다.

오개념 2 — 항등관계는 대칭이자 반대칭

항등관계 $I_A = \{(a,a)\}$ 는 $(a,b) \in R$ 이면 자동으로 $a=b$ 이므로 대칭의 조건과 반대칭의 조건을 모두 만족한다. '대칭 $\cap$ 반대칭' 이 비어 있지 않다는 좋은 예이다.

증명

정리하면 — 네 가지 경우가 모두 가능하다.

대칭 ○ 반대칭 ○ : 항등관계

대칭 ○ 반대칭 × : 친구 관계 (서로 다른 두 친구가 양방향)

대칭 × 반대칭 ○ : $a < b$ 관계, $a \leq b$ 관계

대칭 × 반대칭 × : (1,2), (2,1), (2,3) 같은 관계

추이관계 — 징검다리로 이어짐

a 에서 b 로, b 에서 c 로 관계가 이어지면 a 에서 c 로도 직접 관계가 있을 때, 추이관계라 한다.

정의 — 추이관계

$$(a, b) \in R \wedge (b, c) \in R \implies (a, c) \in R$$

추이성은 '징검다리 건너뛰기' 이다. 중간 정점 b 를 거쳐 갈 수 있으면, 직접 가는 길도 반드시 있어야 한다.

유향 그래프에서의 특징

$a \rightarrow b$ 와 $b \rightarrow c$ 화살표가 있으면, $a \rightarrow c$ 화살표도 반드시 존재한다 (지름길이 항상 있다).

AI 연결 — 추이성과 멀티홉 추론

'A 는 B 의 조상, B 는 C 의 조상 $\implies$ A 는 C 의 조상' 처럼, 추이관계는 멀티홉 추론을 한 칸으로 줄여 준다. 추이폐포(5.6절)는 이 지름길을 모두 추가해 R_∞ 를 만든다.

예제 5-29 — 추이관계 판별

문제

$A = \{1, 2, 3\}$ 위의 다음 관계가 추이관계인지 판별하라.

$R8 = \{(1,2), (2,3), (1,3)\}$ $R9 = \{(1,2), (2,3)\}$

증명

추이관계이려면 (a,b) 와 (b,c) 가 있을 때마다 (a,c) 도 있어야 한다.

R8: $(1,2)$ 와 $(2,3)$ 이 있다 $\Rightarrow (1,3)$ 이 필요 $\Rightarrow (1,3)$ 있음 ○
그 밖에 이어지는 쌍이 없으므로 $\Rightarrow$ 추이관계이다.

R9: $(1,2)$ 와 $(2,3)$ 이 있다 $\Rightarrow (1,3)$ 이 필요 $\Rightarrow (1,3)$ 없음 ×
따라서 $\Rightarrow$ 추이관계가 아니다.

추이성 판별의 요령: 이어지는 쌍 $(a,b), (b,c)$ 를 모두 찾고, 각각에 대해 지름길 (a,c) 가 있는지 확인한다.

예제 5-30 — 부등호 관계의 추이성

문제

정수 집합 위의 관계 $a < b$ 와 $a \leq b$ 가 각각 추이관계인지 판별하라.

증명

관계 $a < b$ 의 경우:

$a < b$ 이고 $b < c$ 이면, 부등식의 성질에 의해 $a < c$ 이다.

모든 정수 a, b, c 에 대해 성립하므로 $\Rightarrow a < b$ 는 추이관계이다.

관계 $a \leq b$ 의 경우:

$a \leq b$ 이고 $b \leq c$ 이면, 마찬가지로 $a \leq c$ 이다.

따라서 $\Rightarrow a \leq b$ 도 추이관계이다.

이처럼 무한집합 위의 관계는 순서쌍을 일일이 셀 수 없으므로, '임의의 a, b, c 에 대해' 논증으로 판별한다. 이 일반 논증이 바로 Lean 4 형식 증명과 똑같은 방식이다.

예제 5-31 — 여러 성질 종합 판별

문제

$A = \{1,2,3\}$ 위의 $R = \{(1,1), (1,2), (2,1), (2,2), (3,3)\}$ 의 반사 · 대칭 · 추이성을 모두 판별하라.

증명

반사성: $(1,1), (2,2), (3,3)$ 이 모두 있다 $\Rightarrow$ 반사관계이다.

대칭성: $(1,2)$ 의 짝 $(2,1)$ 있음. 나머지는 모두 (a,a) 꼴.
모든 쌍이 짝을 이루므로 $\Rightarrow$ 대칭관계이다.

추이성: 이어지는 쌍을 확인한다.

$(1,2), (2,1) \Rightarrow (1,1) \bigcirc$ $(2,1), (1,2) \Rightarrow (2,2) \bigcirc$

$(1,2), (2,2) \Rightarrow (1,2) \bigcirc$ $(2,1), (1,1) \Rightarrow (2,1) \bigcirc \dots$

필요한 지름길이 모두 있으므로 $\Rightarrow$ 추이관계이다.

이 R 은 반사 · 대칭 · 추이를 모두 갖는다 — 곧 동치관계이다. 동치관계는 다음에 자세히 본다.

예제 5-32 — 관계행렬로 성질 판별

$A = \{1,2,3\}$ 위의 관계가 아래 관계행렬로 주어졌다. 세 성질을 관계행렬만으로 판별한다.

	1	2	3
1	1	1	0
2	1	1	0
3	0	0	1

관계행렬 판별 요령 — 반사: 주대각선 1 확인. 대칭: 행렬이 거울 대칭인지 확인. 추이: 부울곱 $M \odot M$ 의 1 인 칸이 원래 M 에서도 1 인지 확인 (M^2 가 M 에 포함되면 추이관계).

증명

반사성: 주대각선이 모두 1
 $\Rightarrow$ 반사관계이다.

대칭성: 주대각선 기준으로
행렬이 거울 대칭
 $\Rightarrow$ 대칭관계이다.

추이성: 지름길 점검 결과
 $\Rightarrow$ 추이관계이다.

추이성의 행렬 판정

관계 R 이 추이관계일 필요충분조건은 $R^2 \subseteq R$, 곧 $M \odot M$ 의 모든 1 이 M 에서도 1 인 것이다.

동치관계 — 같은 것끼리 묶기

반사성 · 대칭성 · 추이성을 모두 갖춘 관계를 동치관계라 한다. 동치관계는 원소들을 '사실상 같은 것끼리' 묶는 역할을 한다.

정의 — 동치관계

관계 R 이 반사적이고, 대칭적이고, 추이적이면 R 을 동치관계라 한다. 세 성질 중 하나라도 빠지면 동치관계가 아니다.

증명

세 성질이 함께 작동하는 방식:

반사성 — 자기 자신과 같다. 대칭성 — a 가 b 와 같으면 b 도 a 와 같다.

추이성 — a 가 b 와 같고 b 가 c 와 같으면 a 도 c 와 같다.

동치관계는 '같음' 의 일반화

'=' (같다) 자체가 동치관계의 대표이다. 동치관계는 '같다' 의 조건을 추상화하여, 다른 기준으로도 '같다고 볼 수 있게' 해 준다. 합동(나머지가 같음), 닮음 등이 모두 동치관계이다.

예제 5-33 — 합동관계는 동치관계

문제 정수 집합 위의 관계를 'a 와 b 를 3 으로 나눈 나머지가 같다' 로 정의한다. 이 관계가 동치 관계임을 보여라.

증명

a 를 3 으로 나눈 나머지를 $r(a)$ 라 하자. 관계는 $r(a) = r(b)$ 이다.

반사성: $r(a) = r(a)$ 는 항상 참 $\Rightarrow (a, a) \in R$.

대칭성: $r(a) = r(b)$ 이면 $r(b) = r(a)$ $\Rightarrow (a, b) \in R$ 이면 $(b, a) \in R$.

추이성: $r(a) = r(b)$ 이고 $r(b) = r(c)$ 이면 $r(a) = r(c)$
 $\Rightarrow (a, b) \in R$ 이고 $(b, c) \in R$ 이면 $(a, c) \in R$.

세 성질을 모두 만족하므로 이 관계는 동치관계이다. ■

이 관계를 '법 3 에 대한 합동' 이라 하고 $a \equiv b \pmod{3}$ 로 쓴다. 4장에서 배운 합동이 곧 동치관계이다.

예제 5-34 — 동치관계가 아닌 예

문제

정수 집합 위의 관계 $a \leq b$ 가 동치관계인지 판별하라.

증명

세 성질을 차례로 점검한다.

반사성: $a \leq a$ 는 항상 참 $\Rightarrow$ 반사관계 ○

추이성: $a \leq b, b \leq c$ 이면 $a \leq c$ $\Rightarrow$ 추이관계 ○

대칭성: $1 \leq 2$ 이지만 $2 \leq 1$ 은 거짓 $\Rightarrow$ 대칭관계 ✕

대칭성이 깨지므로 $a \leq b$ 는 동치관계가 아니다.

$a \leq b$ 는 반사 · 추이 · 반대칭을 갖춘다. 이런 관계를 부분순서관계라 하며, 7장의 주제이다. 동치관계와 부분순서관계는 '대칭이나 반대칭이나' 에서 갈린다.

STUDENT TRY

'a 와 b 는 같은 부호다' 라는 관계는 동치관계인가? 세 성질을 직접 점검해 보라.

동치류 — 동치관계가 만드는 묶음

동치관계가 주어지면, 각 원소와 '같다고 묶이는' 원소들의 집합을 생각할 수 있다. 이것이 동치류이다.

정의 — 동치류

$$[a]_R = \{ x \in A : (x, a) \in R \}$$

원소 a 의 동치류 $[a]$ 는 a 와 동치관계에 있는 모든 원소의 집합이다. a 자신도 반사성에 의해 $[a]$ 에 속한다.

증명

동치류의 핵심 성질: 두 동치류 $[a]$ 와 $[b]$ 는 완전히 같거나, 아니면 서로소이다.
곧 동치류들은 집합 A 를 겹치지 않게 조각낸다 — 이를 분할이라 한다.

AI 응용 — 동치류는 군집

동치류는 데이터를 겹치지 않는 군집(cluster)으로 나눈다. 중복 문서 제거, 사용자 세분화, 이미지의 유사 픽셀 묶기 등이 모두 동치류로 데이터를 분할하는 작업이다.

예제 5-35 — 동치류 구하기

문제

정수 집합에서 법 3 합동관계 $a \equiv b \pmod{3}$ 의 동치류를 모두 구하라.

증명

정수를 3 으로 나눈 나머지에 따라 묶는다.

$[0] = \{\dots, -6, -3, 0, 3, 6, \dots\}$ 나머지가 0 인 정수

$[1] = \{\dots, -5, -2, 1, 4, 7, \dots\}$ 나머지가 1 인 정수

$[2] = \{\dots, -4, -1, 2, 5, 8, \dots\}$ 나머지가 2 인 정수

동치류는 $[0], [1], [2]$ 의 셋뿐이다. 예컨대 $[3]$ 은 $[0]$ 과 같은 집합이다.
세 동치류는 서로소이고, 그 합집합이 정수 집합 전체이다. 곧 $\{[0], [1], [2]\}$ 는 정수 집합의 분할이다.
이 분할이 4장에서 배운 법 3 잉여계 Z_3 와 정확히 대응한다.

동치관계와 분할 — 동전의 양면

동치관계 ↔ 분할 대응 정리

집합 A 위의 동치관계는 A 의 분할을 하나 만들고(동치류들), 거꾸로 A 의 임의의 분할은 동치관계를 하나 만든다(같은 조각에 있으면 관계 있음). 둘은 일대일로 대응한다.

증명

분할의 정의 — 집합 A 를 다음 세 조건의 부분집합 모임으로 나눈 것:

- (1) 각 조각은 공집합이 아니다.
- (2) 서로 다른 두 조각은 서로소이다.
- (3) 모든 조각의 합집합은 A 전체이다.

왜 이 대응이 강력한가

'관계로 묶는 것'과 '집합을 조각내는 것'이 같은 일임을 뜻한다. 그래서 군집화 알고리즘은 동치관계를 설계하는 일과 같고, 데이터 분할 문제는 동치류 찾기 문제로 환원된다.

Lean 4 — 관계의 성질을 정의하기

다섯 가지 성질을 Lean 4 의 술어(`Prop` 을 돌려주는 정의)로 옮긴다. 수학 정의와 글자 그대로 닮아 있다.

```
variable {A : Type}

def Refl (R : A → A → Prop) : Prop := ∀ a, R a a
def Symm (R : A → A → Prop) : Prop := ∀ a b, R a b → R b a
def Trans (R : A → A → Prop) : Prop :=
  ∀ a b c, R a b → R b c → R a c
def Antisymm (R : A → A → Prop) : Prop :=
  ∀ a b, R a b → R b a → a = b

-- 동치관계: 세 성질을 그리고( $\wedge$ )로 묶는다
def Equiv (R : A → A → Prop) : Prop :=
  Refl R ∧ Symm R ∧ Trans R
```

lean

Mathlib 에는 `Reflexive`, `Symmetric`, `Transitive`, `Equivalence` 가 이미 있다. 여기서는 학습을 위해 직접 정의한다.

수학 ↔ Lean — 반사성 증명

수학 추론

증명

주장: 자연수 위의 관계
 $a \leq b$ 는 반사관계이다.

증명:
임의의 자연수 a 를 잡는다.

자연수의 부등식 성질에
의해 $a \leq a$ 는 항상
참이다.

따라서 (a, a) 가 관계에
속하므로, 이 관계는
반사관계이다. ■

Lean 4 증명

lean

```
def Le : Nat → Nat → Prop :=  
  fun a b => a ≤ b  
  
-- Le 는 반사관계  
example : Refl Le := by  
  -- 목표:  $\forall a, Le\ a\ a$   
  intro a          -- a 를 잡음  
  show a ≤ a        -- 정의를 펼침  
  omega             -- 부등식 닫기
```

수학의 '임의의 a 를 잡는다' 가 Lean 의 `intro a` 이다. ' $\forall$ 를 증명하려면 대표 원소 하나를 잡는다' 가 핵심.

수학 ↔ Lean — 대칭성 증명

수학 추론

증명

주장: 관계 '같다'(=)는
대칭관계이다.

증명:

임의의 a, b 를 잡고,
 $a = b$ 라 가정한다.

보여야 할 것은 $b = a$.

가정 $a = b$ 의 좌우를
바꿔 쓰면 곧 $b = a$
이다. ■

Lean 4 증명

lean

```
def EqR (A : Type) :  
  A → A → Prop :=  
  fun a b => a = b  
  
example {A : Type} :  
  Symm (EqR A) := by  
  -- ∀ a b, a=b → b=a  
  intro a b h      -- 가정 h : a=b  
  show b = a  
  rw [h]            -- a 를 b 로 바꿈  
  -- 목표가 b=b 가 되어 닫힘
```

`rw [h]` 는 가정 $h : a = b$ 를 써서 목표의 a 를 b 로 바꾼다. 목표 $b = a$ 가 $b = b$ 로 바뀌어 자동으로 닫힌다.

수학 ↔ Lean — 추이성 증명

수학 추론

증명

주장: 자연수 위의 $a \leq b$ 는 추이관계이다.

증명:

임의의 a, b, c 를 잡고
 $a \leq b$ 와 $b \leq c$ 를
가정한다.

부등식의 추이성에 의해
 $a \leq b$ 이고 $b \leq c$ 이면
 $a \leq c$ 이다. ■

Lean 4 증명

lean

```
-- Le 는 추이관계
example : Trans Le := by
  --  $\forall a b c,$ 
  --  $Le\ a\ b \rightarrow Le\ b\ c \rightarrow Le\ a\ c$ 
  intro a b c hab hbc
  -- hab :  $a \leq b$ 
  -- hbc :  $b \leq c$ 
  show a ≤ c
  omega      -- 두 부등식 결합
```

가정이 둘이면 intro 로 둘 다 받는다 (hab, hbc). omega 는 두 부등식을 결합해 $a \leq c$ 를 끌어낸다.

Lean 4 — '같다'가 동치관계임을 증명

세 성질을 한꺼번에 묶어 동치관계임을 증명한다. constructor 로 '그리고($\wedge$)' 목표를 차례로 쪼갬다.

```
example {A : Type} : Equiv (EqR A) := by
  -- 목표: Refl  $\wedge$  Symm  $\wedge$  Trans
  constructor
  • intro a          -- 반사성
    rfl
  • constructor
    • intro a b h      -- 대칭성
      rw [h]
    • intro a b c hab hbc -- 추이성
      rw [hab, hbc]
```

lean

constructor 는 ' $A \wedge B$ ' 목표를 두 부분 목표로 나눈다. 세 성질이므로 한 번 더 constructor 를 써서 쪼갬다.

Lean 4 — 반대칭성 증명

자연수 위의 $a \leq b$ 가 반대칭관계임을 증명한다. 이는 7장 부분순서관계의 토대가 되는 성질이다.

```
example : Antisymm Le := by
  -- 목표:  $\forall a b, Le\ a\ b \rightarrow Le\ b\ a \rightarrow a = b$ 
  intro a b hab hba
  -- hab :  $a \leq b$ , hba :  $b \leq a$ 
  -- Le 의 정의를 펼쳐 omega 가 보도록 한다
  unfold Le at hab hba
  show a = b
  omega      --  $a \leq b$  이고  $b \leq a$  이면  $a = b$ 
```

lean

unfold Le 는 정의를 펼쳐 hab, hba 를 순수한 부등식으로 만든다. 그러면 omega 가 ' $a \leq b$ 이고 $b \leq a \Rightarrow a = b$ ' 를 끌어낸다.

Python — 관계의 성질 자동 판별

관계를 순서쌍 집합으로 두면, 세 성질을 코드로 자동 판별할 수 있다. 정의를 그대로 옮기면 된다.

python

```
def is_reflexive(A, R):
    return all((a, a) in R for a in A)

def is_symmetric(R):
    return all((b, a) in R for (a, b) in R)

def is_transitive(R):
    return all((a, c) in R
               for (a, b) in R for (x, c) in R if b == x)

A = {1, 2, 3}
R = {(1,1), (1,2), (2,1), (2,2), (3,3)}
equiv = (is_reflexive(A, R) and is_symmetric(R)
         and is_transitive(R))    # True – 동치관계
```

세 함수는 각 성질의 수학 정의를 `all(...)` 한 줄로 옮긴 것이다. `equiv` 가 `True` 이면 동치관계이다.

AI 응용 — 동치관계와 데이터 군집화

동치관계가 데이터를 겹치지 않는 군집으로 나눈다는 성질은 현대 AI 파이프라인 곳곳에서 쓰인다.

중복 제거

'두 문서의 해시가 같다' 는 동치관계. 동치류 하나당 문서 하나만 남겨 학습 데이터를 정제한다.

사용자 세분화

'같은 행동 패턴' 동치관계로 사용자를 군집으로 나눠 군집별 추천 전략을 세운다.

RAG 문서 묶기

'같은 주제' 동치관계로 문서를 묶어, 검색 시 군집 단위로 다양성을 확보한다.

이미지 분할

'연결된 같은 색 영역' 동치관계로 픽셀을 동치류로 묶어 객체를 분리한다.

5.4 요약 — 관계의 성질

반사 · 비반사

모든 (a,a) 있음 / 모든 (a,a) 없음.

대칭 · 비대칭 · 반대칭

양방향 / 결코 양방향 아님 / 양방향이면 $a=b$.

추이

$(a,b), (b,c)$ 있으면 (a,c) 도 있음. 지름길 보장.

동치관계

반사 + 대칭 + 추이. '같음' 의 일반화.

동치류 · 분할

동치류들이 집합을 겹치지 않게 조각냄.

PART

5.5

역관계와 합성관계

관계의 방향을 뒤집은 역관계, 관계 밖을 모은 여관계,
그리고 두 관계를 이어 붙인 합성관계를 다룬다.

역관계 — 화살표를 뒤집기

관계 R 의 모든 순서쌍에서 두 성분의 자리를 바꾸면 새로운 관계가 된다. 이를 역관계라 한다.

정의 — 역관계

$$R^{-1} = \{ (b, a) : (a, b) \in R \}$$

$(a, b) \in R$ 이면 $(b, a) \in R^{-1}$ 이다. 유향 그래프로 보면 모든 화살표의 방향을 거꾸로 뒤집은 것이다.

관계행렬에서의 역관계

역관계 R^{-1} 의 관계행렬은 R 의 관계행렬을 전치(transpose)한 것이다 — 행과 열을 맞바꾼다.

일상 비유

'a 가 b 의 부모다' 의 역관계는 'b 가 a 의 자식이다' 이다. 'a 가 b 를 팔로우한다' 의 역관계는 'b 가 a 에게 팔로우당한다' 이다. 역관계는 같은 사실을 반대편 시점에서 본 것이다.

예제 5-36 — 역관계 구하기

문제

$A = \{1,2,3\}$ 에서 $B = \{a,b\}$ 로의 관계 $R = \{(1,a), (1,b), (2,b), (3,a)\}$ 의 역관계 R^{-1} 를 구하라.

증명

R 의 각 순서쌍에서 두 성분의 자리를 맞바꾼다.

$$(1,a) \rightarrow (a,1), (1,b) \rightarrow (b,1), (2,b) \rightarrow (b,2), (3,a) \rightarrow (a,3)$$

$$R^{-1} = \{(a,1), (b,1), (b,2), (a,3)\}$$

R 은 A 에서 B 로의 관계였지만, R^{-1} 는 B 에서 A 로의 관계이다. 출발 집합과 도착 집합도 맞바뀐다.

STUDENT TRY

R^{-1} 의 역관계 $(R^{-1})^{-1}$ 를 구해 보라. 원래의 R 과 같은가? 왜 그런지 한 문장으로 설명하라.

예제 5-37 — 역관계의 관계행렬

관계행렬로 주어진 관계 R 의 역관계는 행렬을 전치하여 얻는다. 행과 열을 맞바꾼다.

	M_R		
	1	2	3
1	1	0	1
2	1	1	0
3	0	1	0

	$M_{R^{-1}}$ (전치)		
	1	2	3
1	1	1	0
2	0	1	1
3	1	0	0

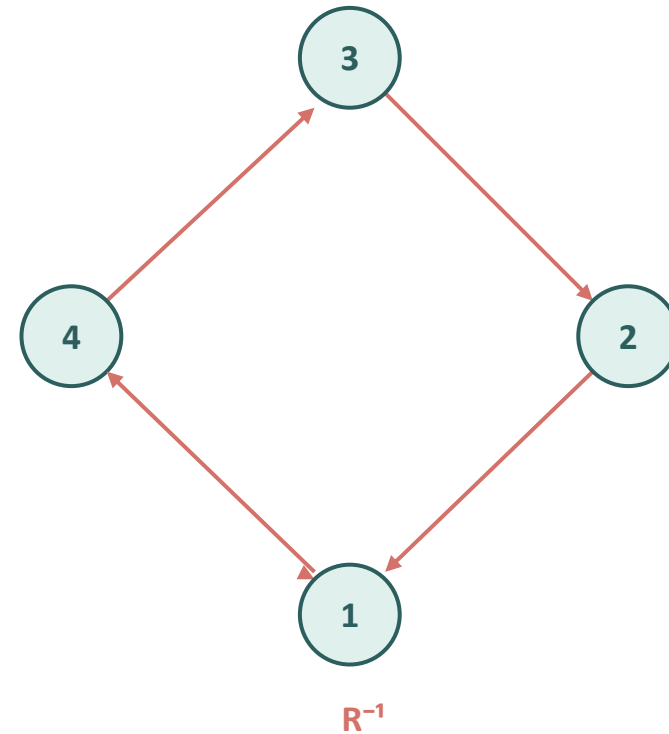
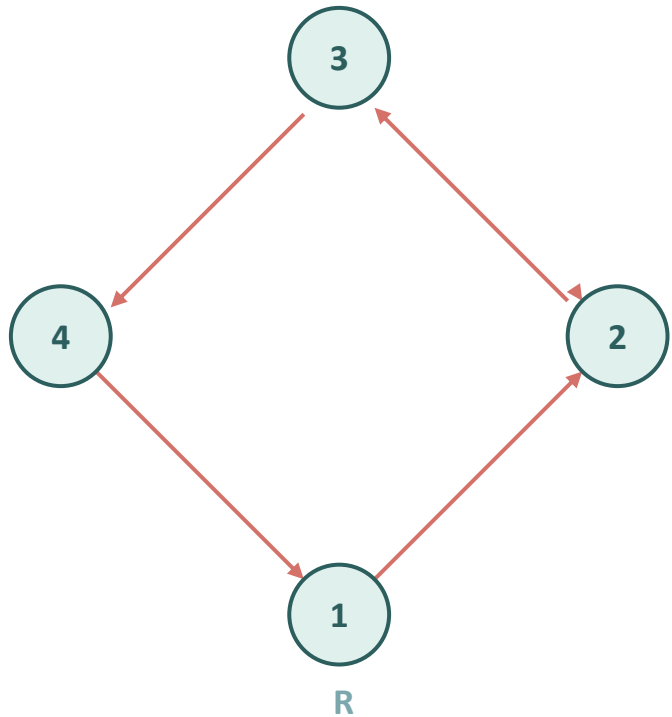
M_R 의 i 행 j 열 성분이 $M_{R^{-1}}$ 에서는 j 행 i 열로 옮겨 간다. 주대각선을 접는 선으로 삼아 뒤집은 것이다.

관계가 대칭관계이면 $M_R = M_{R^{-1}}$ 이다 — 전치해도 변하지 않기 때문이다. 이 사실은 곧 다시 다룬다.

예제 5-38 — 역관계와 유형 그래프

문제

$A = \{1, 2, 3, 4\}$ 위의 $R = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$ 의 역관계를 유형 그래프로 나타내라.



R 은 시계 방향 순환, R^{-1} 은 반시계 방향 순환이다. 정점은 그대로 두고 모든 화살표의 방향만 뒤집었다.

역관계의 성질

성질 1 — 역관계의 역관계

$(R^{-1})^{-1} = R$. 방향을 두 번 뒤집으면 원래대로 돌아온다.

성질 2 — 역관계와 대칭관계

관계 R 이 대칭관계일 필요충분조건은 $R^{-1} = R$ 이다. 대칭관계는 뒤집어도 변하지 않는 관계이다.

증명

성질 2 의 증명 (개념):

$(\Rightarrow)$ R 이 대칭이면, $(a,b) \in R$ 마다 $(b,a) \in R$ 이다. 곧 R^{-1} 의 원소가 모두 R 에 있고, 그 역도 같으므로 $R^{-1} = R$.

$(\Leftarrow)$ $R^{-1} = R$ 이면, $(a,b) \in R \Rightarrow (b,a) \in R^{-1} = R$. 곧 R 은 대칭관계이다. ■

필요충분조건이므로 양방향($\Rightarrow$, $\Leftarrow$)을 모두 증명해야 한다. 이 구조는 Lean 4 의 constructor 와 똑같다.

여관계 — 관계의 바깥쪽

곱집합 $A \times B$ 전체에서 관계 R 에 들지 않은 순서쌍을 모으면 여관계가 된다.

정의 — 여관계

$$\overline{R} = (A \times B) - R$$

$(a, b) \in R^-$ 일 필요충분조건은 $(a, b) \notin R$ 이다. 여관계는 'R 이 아닌 모든 관계' 이다.

관계행렬에서의 여관계

여관계 R^- 의 관계행렬은 R 의 관계행렬에서 0 과 1 을 모두 뒤집은 것이다 ($0 \leftrightarrow 1$).

역관계와 여관계는 전혀 다르다

역관계 R^{-1} 는 '방향을 뒤집은' 관계, 여관계 R^- 는 'R 에 없는' 관계이다. 이름이 비슷하나 하는 일이 완전히 다르다. R^{-1} 는 전치, R^- 는 0-1 반전이다.

예제 5-39 — 여관계 구하기

문제

$A = \{1,2\}$ 에서 $B = \{a,b\}$ 로의 관계 $R = \{(1,a), (2,b)\}$ 의 여관계 R^- 를 구하라.

증명

먼저 곱집합 $A \times B$ 전체를 적는다.

$$A \times B = \{(1,a), (1,b), (2,a), (2,b)\}$$

여관계는 $A \times B$ 에서 R 의 원소를 뺀 것이다.

$$R^- = (A \times B) - R = \{(1,b), (2,a)\}$$

R 과 R^- 는 서로 겹치지 않고, 둘을 합치면 곱집합 $A \times B$ 전체가 된다. 곧 R 과 R^- 는 $A \times B$ 의 분할을 이룬다.

예제 5-40 — 여관계의 관계행렬

여관계의 관계행렬은 원래 행렬의 0 과 1 을 모두 뒤집어 얻는다.

M_R			
	1	2	3
1	1	0	1
2	0	1	0
3	1	1	0

M_R (0-1 반전)			
	1	2	3
1	0	1	0
2	1	0	1
3	0	0	1

M_R 에서 1 이던 칸은 0 으로, 0 이던 칸은 1 로 바뀐다. 두 행렬을 같은 자리끼리 더하면 모든 칸이 1 이다.

여관계의 쓰임

추천 시스템에서 '아직 보지 않은 상품' 은 '본 상품' 관계의 여관계이다. 추천 후보는 대개 여관계에서 찾는다. 이미 가진 관계의 바깥쪽을 보는 일은 생각보다 자주 필요하다.

합성관계 — 두 관계를 이어 붙이기

관계 R 로 한 칸, 이어서 관계 S 로 한 칸 가는 길을 묶으면 합성관계가 된다.

정의 — 합성관계

$$S \circ R = \{ (a, c) : \exists b [(a, b) \in R \wedge (b, c) \in S] \}$$

$(a, c) \in S \circ R$ 일 필요충분조건은, 중간 원소 b 가 있어 $(a, b) \in R$ 이고 $(b, c) \in S$ 인 것이다.

표기 순서에 주의

$S \circ R$ 은 'R 을 먼저, S 를 나중에' 적용한다. 함수 합성과 똑같이 오른쪽 관계가 먼저이다. 교재에 따라 $R \circ S$ 로 쓰기도 하므로, 어느 쪽을 먼저 적용하는지 늘 확인한다.

관계의 거듭제곱 R^2 은 $R \circ R$, 곧 자기 자신과의 합성이다. 5.3절의 경로가 바로 이 합성이었다.

예제 5-41 — 합성관계 구하기

문제

$R = \{(1,2), (2,3), (3,4)\}$ 이고 $S = \{(2,a), (3,b), (4,c)\}$ 일 때 합성관계 $S \circ R$ 을 구하라.

증명

$(a, c) \in S \circ R$ 이려면 $(a,b) \in R$ 이고 $(b,c) \in S$ 인 중간 원소 b 가 있어야 한다.

$$(1,2) \in R \text{ 와 } (2,a) \in S \Rightarrow (1,a) \in S \circ R$$

$$(2,3) \in R \text{ 와 } (3,b) \in S \Rightarrow (2,b) \in S \circ R$$

$$(3,4) \in R \text{ 와 } (4,c) \in S \Rightarrow (3,c) \in S \circ R$$

$$S \circ R = \{(1,a), (2,b), (3,c)\}$$

R 의 순서쌍 (x,y) 마다, S 에서 첫 성분이 y 인 순서쌍 (y,z) 를 찾아 (x,z) 를 만든다. 중간 원소 y 가 ' R 의 끝' 과 ' S 의 시작' 을 이어 준다.

예제 5-42 — 합성관계의 관계행렬

합성관계 $S \circ R$ 의 관계행렬은 M_R 과 M_S 의 부울곱이다. 길이 2 경로를 세던 5.3절 정리 5-2 와 같은 원리이다.

	M_R		
	1	2	3
1	0	1	0
2	0	0	1
3	1	0	0

	M_S		
	1	2	3
1	1	0	0
2	0	1	0
3	0	0	1

	$M_S \odot M_R$		
	1	2	3
1	0	1	0
2	0	0	1
3	1	0	0

여기서 S 는 항등관계이므로 $S \circ R = R$ 이 된다 — 항등관계와의 합성은 원래 관계를 그대로 둔다. 항등 관계는 합성에 대한 '단위원' 역할을 한다.

주의: 부울곱의 순서는 $M_S \odot M_R$ 이다 ($S \circ R$ 에서 R 이 먼저이므로 행렬 곱에서는 M_R 이 오른쪽).

합성은 순서를 바꾸면 달라진다

관계의 합성은 일반적으로 가환이 아니다. 곧 $S \circ R$ 과 $R \circ S$ 는 보통 서로 다른 관계이다.

비가환성

$$S \circ R \neq R \circ S \quad (\text{!!!!!!!!!!})$$

증명

반례: $R = \{(1,2)\}$, $S = \{(2,3)\}$ 이라 하자.

$S \circ R$: $(1,2) \in R, (2,3) \in S \Rightarrow (1,3) \in S \circ R$. 곧 $S \circ R = \{(1,3)\}$.

$R \circ S$: $(b,c) \in S$ 이고 $(c,d) \in R$ 인 중간 원소가 없다. 곧 $R \circ S = \emptyset$.

따라서 $S \circ R = \{(1,3)\} \neq \emptyset = R \circ S$ 이다.

단 하나의 반례만 보여도 ' $S \circ R = R \circ S$ 가 항상 성립한다' 는 거짓임이 증명된다. 보편 명제를 깨는 데는 반례 하나면 충분하다.

예제 5-43 — $S \circ R$ 과 $R \circ S$ 비교

문제

$R = \{(1,1), (1,2), (2,3)\}$ 이고 $S = \{(1,1), (2,2), (2,3)\}$ 일 때 $S \circ R$ 과 $R \circ S$ 를 각각 구하고 비교하라.

증명

$S \circ R$ (R 먼저, S 나중):

$$(1,1)R, (1,1)S \Rightarrow (1,1)$$

$$(1,2)R, (2,2)S \Rightarrow (1,2)$$

$$(1,2)R, (2,3)S \Rightarrow (1,3)$$

$$S \circ R = \{(1,1), (1,2), (1,3)\}$$

증명

$R \circ S$ (S 먼저, R 나중):

$$(1,1)S, (1,1)R \Rightarrow (1,1)$$

$$(1,1)S, (1,2)R \Rightarrow (1,2)$$

$$(2,3)S, (3,?)R \text{ 없음}$$

$$R \circ S = \{(1,1), (1,2)\}$$

$S \circ R$ 에는 $(1,3)$ 이 있지만 $R \circ S$ 에는 없다. 곧 $S \circ R \neq R \circ S$ — 합성의 비가환성을 다시 확인한다.

Lean 4 — 역관계와 여관계 정의

역관계는 인수의 순서를 바꾸고, 여관계는 명제를 부정한다. 두 정의 모두 한 줄이다.

```
variable {A : Type}

-- 역관계: a 와 b 의 자리를 바꾼다
def Inv (R : A → A → Prop) : A → A → Prop :=
  fun a b => R b a

-- 여관계: 관계의 부정
def Compl (R : A → A → Prop) : A → A → Prop :=
  fun a b => ¬ R a b
```

lean

수학의 ' $R^{-1} = \{(b,a) : (a,b) \in R\}$ ' 가 ' $\text{fun } a \ b \Rightarrow R \ b \ a$ ' 와, ' $R^{-} = R$ 의 바깥' 이 ' $\text{fun } a \ b \Rightarrow \neg R \ a \ b$ ' 와 대응한다.

수학 $\leftrightarrow$ Lean — 역관계의 역관계는 자기 자신

수학 추론

증명

주장: $(R^{-1})^{-1} = R$.

증명:

임의의 a, b 를 잡는다.

$(R^{-1})^{-1}$ 에서 (a,b) 는
 R^{-1} 에서 (b,a) 이고,
이는 다시 R 에서
 (a,b) 이다.

곧 $(a,b) \in (R^{-1})^{-1}$ 와
 $(a,b) \in R$ 이 같은 말이다.
방향을 두 번 뒤집으면
제자리이다. ■

Lean 4 증명

lean

```
-- (R-1)-1 와 R 은 같다
example (R : A → A → Prop)
  (a b : A) :
    Inv (Inv R) a b ↔ R a b :=
  by
    -- Inv (Inv R) a b
    --   = Inv R b a
    --   = R a b
    -- 정의만 펼치면 양변이 같다
  rfl
```

`Inv` 를 두 번 펼치면 양변이 글자 그대로 $R a b$ 가 된다. 그래서 `rfl` 한 줄로 증명이 닫힌다.

수학 $\leftrightarrow$ Lean — 대칭관계는 역관계가 자기 자신

수학 추론

증명

주장: R 이 대칭관계이면
 R^{-1} 의 원소는 모두
 R 의 원소이다.

증명:

$(a,b) \in R^{-1}$ 라 하자.
역관계의 정의에 의해
 $(b,a) \in R$ 이다.

R 이 대칭이므로
 $(b,a) \in R$ 에서 $(a,b) \in R$
을 얻는다. ■

Lean 4 증명

lean

```
-- Symm 은 5.4 의 정의를 사용
example (R : A → A → Prop)
  (hsymm : Symm R)
  (a b : A) :
    Inv R a b → R a b := by
  intro h
  -- h : Inv R a b, 곧 R b a
  -- hsymm b a : R b a → R a b
  exact hsymm b a h
```

$\text{Inv } R \ a \ b$ 는 정의상 $R \ b \ a$ 이다. 대칭성 hsymm 을 b, a 에 적용하면 $R \ a \ b$ 를 얻어 증명이 닫힌다.

Lean 4 — 합성의 비가환성을 반례로

$S \circ R \neq R \circ S$ 를 Lean 으로 보인다. 구체적인 R, S 를 정하고, $S \circ R$ 에는 있지만 $R \circ S$ 에는 없는 쌍을 제시한다.

```
def R1 : Nat → Nat → Prop := fun a b => a = 1 ∧ b = 2
def S1 : Nat → Nat → Prop := fun a b => a = 2 ∧ b = 3

-- (1,3) 은 S∘R 에 있다: 중간 원소 2 를 거친다
example : Comp R1 S1 1 3 :=
  ⟨2, ⟨rfl, rfl⟩, ⟨rfl, rfl⟩⟩

-- (1,3) 은 R∘S 에는 없다
example : ¬ Comp S1 R1 1 3 := by
  rintro ⟨b, ⟨h1, h2⟩, h3⟩ -- h1 : (1:Nat) = 2
  omega                  -- 1 = 2 은 모순
```

lean

$S \circ R$ 에는 $(1,3)$ 이 있고 $R \circ S$ 에는 없으므로 두 관계는 다르다. `rintro` 로 가정을 분해하면 $1 = 2$ 라는 모순이 드러난다.

Python — 역 · 여 · 합성 관계 계산

세 연산은 모두 순서쌍 집합 위의 간단한 조작이다. 각 정의를 한 줄씩 그대로 옮긴다.

```
from itertools import product

def inverse(R):                                # 역관계
    return {(b, a) for (a, b) in R}

def complement(A, B, R):                       # 여관계
    return set(product(A, B)) - set(R)

def compose(R, S):                             # 합성관계 S∘R
    return {(a, c) for (a, b) in R
              for (x, c) in S if b == x}

R = {(1,2), (2,3), (3,4)}
S = {(2,'a'), (3,'b'), (4,'c')}
print(compose(R, S))    # {(1,'a'), (2,'b'), (3,'c')}
```

python

5.5 요약 — 역관계와 합성관계

역관계 R^{-1}

방향을 뒤집음. 관계행렬은 전치. $(R^{-1})^{-1} = R$.

여관계 R^{-}

$(A \times B) - R$. 관계행렬은 0-1 반전.

대칭관계

R 이 대칭일 필요충분조건은 $R^{-1} = R$.

합성관계 $S \circ R$

R 한 칸 + S 한 칸. 관계행렬은 부울곱.

비가환성

일반적으로 $S \circ R \neq R \circ S$. 반례로 확인.

PART

5.6

연결관계와 와샬 알고리즘

관계에 성질을 최소한으로 더하는 폐포 개념과,
추이폐포를 유한 번에 구하는 와샬 알고리즘을 다룬다.

폐포 — 성질을 최소한으로 채우기

주어진 관계가 어떤 성질(반사 · 대칭 · 추이)을 갖지 못할 때, 순서쌍을 '꼭 필요한 만큼만' 추가해 그 성질을 갖게 만들 수 있다. 이렇게 얻은 가장 작은 관계를 폐포라 한다.

정의 — 폐포

관계 R 에 대해, 성질 P 를 만족하면서 R 을 포함하는 가장 작은 관계를 R 의 P -폐포라 한다. '가장 작은' 이란, P 를 만족하며 R 을 포함하는 어떤 관계에도 포함된다는 뜻이다.

곧 폐포는 두 조건을 만족한다 — (1) 원하는 성질을 갖는다, (2) 그 성질을 갖는 관계 중 가장 작다 (불필요한 순서쌍을 단 하나도 더 넣지 않는다).

왜 '가장 작은' 이 중요한가

성질만 채우면 되는 거라면 곱집합 전체를 답으로 삼아도 된다 — 그러나 그것은 정보를 모두 잃는다. '가장 작은' 이라는 조건이 원래 관계의 정보를 최대한 보존하면서 성질만 더하게 해 준다.

반사폐포 — 빠진 고리만 채우기

정의 — 반사폐포

$$r(R) = R \cup I_A$$

R 의 반사폐포 $r(R)$ 은 R 에 항등관계 I_A 를 합한 것이다. 곧 빠져 있던 모든 (a,a) 만 추가한다.

증명

왜 이것이 '가장 작은' 반사관계인가:

- (1) 모든 (a,a) 를 포함하므로 반사관계이다.
 - (2) 반사관계가 되려면 (a,a) 들이 반드시 필요하므로, 그보다 덜 넣을 수는 없다.
- 따라서 $r(R) = R \cup I_A$ 가 R 을 포함하는 가장 작은 반사관계이다.

예제 5-44

$A = \{1,2,3\}$ 위의 $R = \{(1,2), (2,3), (3,1)\}$ 의 반사폐포를 구하라.

풀이: 빠진 고리 $(1,1), (2,2), (3,3)$ 을 더한다.

$$r(R) = \{(1,1), (1,2), (2,2), (2,3), (3,1), (3,3)\}$$

대칭폐포 — 빠진 반대 방향만 채우기

정의 — 대칭폐포

$$s(R) = R \cup R^{-1}$$

R 의 대칭폐포 $s(R)$ 은 R 에 역관계 R^{-1} 를 합한 것이다. 곧 빠져 있던 반대 방향 화살표만 추가한다.

증명

왜 이것이 '가장 작은' 대칭관계인가:

- (1) $(a,b) \in R$ 이면 $(b,a) \in R^{-1}$ 이므로 $s(R)$ 은 대칭관계이다.
- (2) 대칭관계가 되려면 각 (a,b) 의 짝 (b,a) 가 반드시 필요하므로 더 줄일 수 없다.
따라서 $s(R) = R \cup R^{-1}$ 가 R 을 포함하는 가장 작은 대칭관계이다.

예제 5-45

$A = \{1,2,3\}$ 위의 $R = \{(1,2), (2,3), (3,1)\}$ 의 대칭폐포를 구하라.

풀이: 반대 방향 $(2,1), (3,2), (1,3)$ 을 더한다.

$$s(R) = \{(1,2), (2,1), (2,3), (3,2), (3,1), (1,3)\}$$

추이폐포 — 모든 지름길 채우기

추이폐포는 R 에 '징검다리로 갈 수 있는 모든 지름길' 을 추가해 추이관계로 만든 것이다. 반사 · 대칭 폐포보다 훨씬 어렵다.

정의 — 추이폐포

$$t(R) = R^\infty = \bigcup_{n=1}^{\infty} R^n$$

R 의 추이폐포 $t(R)$ 은 연결관계 R^∞ 와 같다. 곧 '길이 1, 2, 3, ... 인 경로로 닿는 모든 쌍' 의 모임이다.

추이폐포가 어려운 이유

반사폐포는 (a,a) 만, 대칭폐포는 (b,a) 만 더하면 끝난다 — 한 번에 채워진다. 그러나 추이폐포는 지름길을 하나 더하면 그 지름길 때문에 또 새 지름길이 필요해질 수 있다. 이 연쇄가 언제 끝나는지, 어떻게 효율적으로 계산하는지가 5.6절의 핵심 문제이다.

예제 5-46 — 추이폐포 구하기

문제

$A = \{a, b, c, d\}$ 위의 $R = \{(a, b), (b, c), (c, d)\}$ 의 추이폐포 $t(R)$ 을 구하라.

증명

징검다리로 닿을 수 있는 모든 쌍을 찾는다.

길이 1: $(a, b), (b, c), (c, d)$ — R 자체

길이 2: (a, c) $[a \rightarrow b \rightarrow c]$, (b, d) $[b \rightarrow c \rightarrow d]$

길이 3: (a, d) $[a \rightarrow b \rightarrow c \rightarrow d]$

$t(R) = \{(a, b), (a, c), (a, d), (b, c), (b, d), (c, d)\}$

원래 3개이던 순서쌍이 추이폐포에서는 6개가 되었다. 추가된 $(a, c), (a, d), (b, d)$ 가 바로 지름길이다.

STUDENT TRY

이 R 에 (d, a) 를 추가하면 추이폐포는 어떻게 달라지는가? 순환이 생기면 무슨 일이 벌어지는가?

추이폐포를 거듭제곱으로 구하면

추이폐포는 $R^\infty = R \cup R^2 \cup R^3 \cup \dots$ 이다. 유한집합에서는 이 합집합이 유한 번에 멈춘다.

유한집합에서의 추이폐포

$|A| = n$ 이면, 길이가 n 을 넘는 경로는 반드시 정점을 다시 지난다(비둘기집 원리). 따라서 $t(R) = R \cup R^2 \cup \dots \cup R^n$ 으로, n 개 항의 합집합이면 충분하다.

증명

거듭제곱 방식의 계산량:

$R^2, R^3, \dots, R^n$ 을 각각 부울곱으로 구한다 — 부울곱 $n-1$ 회.

부울곱 한 번은 약 n^3 번의 연산이 든다.

전체 계산량은 대략 n^4 에 비례한다.

더 빠른 방법이 필요하다

정점이 1000 개면 n^4 은 1조 번이다. 와샬 알고리즘은 같은 일을 n^3 으로 줄인다 — 1000 개 정점에서 10억 번으로, 1000 배 빠르다.

와샬 알고리즘 — 경유지를 하나씩 허용

와샬 알고리즘의 핵심 발상은 '거쳐 가도 되는 중간 정점' 을 하나씩 늘려 가며 도달 가능성을 갱신하는 것이다.

기본 아이디어

단계 k 에서는 '중간 정점으로 1번부터 k 번까지만 거치는 경로'가 있는지를 행렬 W_k 에 기록한다.

W_0 = 원래 관계행렬 (경유 없음, 직접 간선만)

W_n = 모든 정점을 경유지로 허용 — 이것이 곧 추이폐포의 관계행렬
단계를 0에서 n 까지 올리며 W 를 갱신하면, 마지막에 추이폐포가 완성된다.

증명

k 번째 단계의 갱신 규칙 — 정점 k 를 새 경유지로 허용했을 때:

i 에서 j 로 가는 길이 ($k-1$ 단계까지의 경로로) 이미 있거나,
또는 i 에서 k 로 갈 수 있고 그리고 k 에서 j 로 갈 수 있으면,
 i 에서 j 로 갈 수 있다고 기록한다.

곧 '정점 k 를 중간에 한 번 들르는 우회로'를 새로 인정하는 것이다.

와샬 알고리즘 — 점화식

와샬 점화식

$$W_k[i, j] = W_{k-1}[i, j] \vee (W_{k-1}[i, k] \wedge W_{k-1}[k, j])$$

$W_k[i, j]$ 는 '중간 정점을 1..k 로만 제한했을 때 i 에서 j 로 가는 경로의 존재 여부' 이다.

증명

점화식 읽기:

$$\begin{aligned} W_{\{k-1\}}[i, j] &\leftarrow k \text{ 없이도 이미 갈 수 있던 경우} \\ \vee (W_{\{k-1\}}[i, k] \wedge W_{\{k-1\}}[k, j]) &\leftarrow k \text{ 를 새 경유지로 거쳐 가는 경우} \end{aligned}$$

둘 중 하나라도 참이면 $W_k[i, j] = 1$. OR($\vee$) 와 AND($\wedge$) 만으로 갱신된다.

이 점화식을 $k = 1$ 부터 n 까지 적용하면 W_n 이 추이폐포의 관계행렬이 된다.

예제 5-47 — 와샬 알고리즘 단계별 (1)

$A = \{a,b,c,d\}$ 위의 $R = \{(a,b), (b,c), (c,d)\}$ 에 와샬 알고리즘을 적용한다. 행 · 열 순서는 a,b,c,d 이다.

W_0 (경유 없음)

	a	b	c	d
a	0	1	0	0
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

W_1 (a 경유 허용)

	a	b	c	d
a	0	1	0	0
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

증명

단계 $k = a$: 중간 정점으로 a 를 허용한다.

$$W_1[i,j] = W_0[i,j] \vee (W_0[i,a] \wedge W_0[a,j])$$

a 열 $W_0[*,a]$ 가 모두 0 이다 — a 로 들어오는 간선이 없다.

따라서 a 를 거치는 우회로가 없으므로 $\Rightarrow W_1 = W_0$, 변화 없음.

예제 5-47 — 와살 알고리즘 단계별 (2)

W_2 (a,b 경유)

	a	b	c	d
a	0	1	1	0
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

W_3 (a,b,c 경유)

	a	b	c	d
a	0	1	1	1
b	0	0	1	1
c	0	0	0	1
d	0	0	0	0

증명

단계 $k = b$: b 를 거치는 우회로 — $W_1[i,b] \wedge W_1[b,j]$.

$W_1[a,b]=1$ 이고 $W_1[b,c]=1 \Rightarrow W_2[a,c] = 1$. ($a \rightarrow b \rightarrow c$ 인정)

단계 $k = c$: c 를 거치는 우회로 — $W_2[i,c] \wedge W_2[c,j]$.

$W_2[a,c]=1 \wedge W_2[c,d]=1 \Rightarrow W_3[a,d] = 1$. ($a \rightarrow c \rightarrow d$, 곧 $a \rightarrow b \rightarrow c \rightarrow d$)

$W_2[b,c]=1 \wedge W_2[c,d]=1 \Rightarrow W_3[b,d] = 1$. ($b \rightarrow c \rightarrow d$)

단계 $k = d$: d 행 $W_3[d,*]$ 가 모두 0 이므로 $\Rightarrow W_4 = W_3$, 변화 없음.

예제 5-48 — 와샬 알고리즘의 최종 결과

단계 d 까지 마치면 W_4 가 추이폐포의 관계행렬이다.

$W_4 = t(R)$ 의 관계행렬

	a	b	c	d
a	0	1	1	1
b	0	0	1	1
c	0	0	0	1
d	0	0	0	0

증명

행렬을 순서쌍으로 읽으면:

$$t(R) = \{(a,b), (a,c), (a,d), (b,c), (b,d), (c,d)\}$$

예제 5-46 에서 거듭제곱으로 구한 추이폐포와 정확히 일치한다.

와샬 알고리즘은 거듭제곱 방식과 같은 답을 주지만, 행렬을 단 한 벌만 쓰며 n 번의 단계로 끝낸다.

와샬 알고리즘 — 정당성과 복잡도

정당성

단계 k 를 마쳤을 때 $W_k[i,j] = 1$ 일 필요충분조건은 'i 에서 j 로, 중간 정점을 $1..k$ 안에서만 골라 가는 경로가 존재' 하는 것이다 — k 에 대한 수학적 귀납법으로 증명된다. 따라서 W_n 은 모든 경우를 허용한 도달 가능성, 곧 추이폐포이다.

증명

복잡도 비교 (정점 수 n):

거듭제곱 방식: 부울곱 $n-1$ 회 $\times$ 각 $n^3 \Rightarrow$ 대략 n^4 회 연산.

와샬 알고리즘: $k \cdot i \cdot j$ 세 겹 반복 $\Rightarrow$ 정확히 n^3 회 연산.

추가 메모리: 행렬 한 벌만 제자리에서 갱신 — 별도 행렬이 필요 없다.

와샬 알고리즘의 친척 — 플로이드-와샬

갱신 규칙의 OR · AND 를 '더 짧은 거리 택하기' 로 바꾸면, 모든 정점 쌍 사이의 최단 거리를 구하는 플로이드-와샬 알고리즘이 된다. 내비게이션의 경로 안내, 네트워크 라우팅이 이 알고리즘을 쓴다. 추이폐포와 최단 경로가 사실은 같은 골격의 문제임을 보여 준다.

동치폐포 — 가장 작은 동치관계

관계 R 을 포함하는 가장 작은 동치관계를 동치폐포라 한다. 동치관계는 반사 · 대칭 · 추이를 모두 갖 추어야 하므로, 세 폐포를 차례로 적용한다.

동치폐포의 구성

동치폐포는 반사폐포 $\rightarrow$ 대칭폐포 $\rightarrow$ 추이폐포 순으로 적용해 얻는다. 곧 $e(R) = t(s(r(R)))$ 이다. 순서가 중요하다 — 추이폐포를 마지막에 두어야 한다.

순서를 지켜야 하는 이유

추이폐포를 먼저 하고 대칭폐포를 나중에 하면, 대칭으로 더해진 새 순서쌍 때문에 추이성이 다시 깨질 수 있다. 추이폐포를 가장 마지막에 적용해야 모든 성질이 동시에 유지된다. 반사 $\rightarrow$ 대칭 $\rightarrow$ 추이 순서가 안전하다.

예제 5-49: $R = \{(1,2), (3,4)\}$ 의 동치폐포는 $\{1,2\}$ 와 $\{3,4\}$ 를 각각 한 덩어리로 묶는 동치관계이다.

예제 5-50 ~ 5-52 — 폐포 종합

예제 5-50

$A = \{1,2,3\}$ 위의 $R = \{(1,1), (1,2), (2,3)\}$ 의 반사폐포를 구하라.

풀이: 빠진 $(2,2), (3,3)$ 을 더한다. $r(R) = \{(1,1), (1,2), (2,2), (2,3), (3,3)\}$

예제 5-51

같은 R 의 대칭폐포를 구하라.

풀이: 반대 방향 $(2,1), (3,2)$ 를 더한다. $s(R) = \{(1,1), (1,2), (2,1), (2,3), (3,2)\}$

예제 5-52

같은 R 의 추이폐포를 구하라.

풀이: $(1,2), (2,3)$ 으로 $(1,3)$ 추가. $(1,1), (1,2)$ 로 $(1,2)$ — 이미 있음.

$t(R) = \{(1,1), (1,2), (1,3), (2,3)\}$

세 폐포는 각각 다른 순서쌍을 추가한다. 같은 R 이라도 어떤 성질을 채우느냐에 따라 결과가 전혀 다르다.

Lean 4 — 반사폐포와 대칭폐포 정의

반사폐포는 '원래 관계이거나 같음', 대칭폐포는 '원래 관계이거나 뒤집은 관계' 로 정의한다. 합집합이 Lean 에서는 $\text{OR}(\vee)$ 이다.

```
variable {A : Type}

-- 반사폐포: R 에 항등관계를 합한다
def ReflClosure (R : A → A → Prop) : A → A → Prop :=
  fun a b => R a b ∨ a = b

-- 대칭폐포: R 에 역관계를 합한다
def SymmClosure (R : A → A → Prop) : A → A → Prop :=
  fun a b => R a b ∨ R b a
```

수학의 ' $r(R) = R \cup I_A$ ' 가 ' $R\ a\ b \vee a = b$ ' 와, ' $s(R) = R \cup R^{-1}$ ' 가 ' $R\ a\ b \vee R\ b\ a$ ' 와 대응한다.

lean

수학 ↔ Lean — 반사폐포는 반사적

수학 추론

증명

주장: $r(R)$ 은 반사관계이다.

증명:
임의의 a 를 잡는다.

$r(R)$ 의 정의에서
 $(a,a) \in r(R)$ 이려면
 $(a,a) \in R$ 또는 $a=a$.

$a = a$ 는 항상 참이다.
오른쪽 선택지가 참이므로
 $(a,a) \in r(R)$. ■

Lean 4 증명

lean

```
-- r(R) 은 반사관계
example (R : A → A → Prop) :
  ∀ a, ReflClosure R a a :=
  by
    intro a
    -- 목표: R a a ∨ a = a
    right      -- 오른쪽 택함
    rfl        -- a = a 달기
```

목표가 ' $A \vee B$ ' 일 때 `right` 는 오른쪽 B 를 증명하겠다고 택한다 (왼쪽은 `left`). 여기서는 $a = a$ 가 쉽다.

Lean 4 — 폐포는 원래 관계를 포함한다

폐포의 첫 조건은 'R 을 포함한다' 이다. R 의 순서쌍이 모두 폐포에 들어 있음을 증명한다.

```
-- R 의 원소는 모두 반사폐포에 들어 있다
example (R : A → A → Prop) (a b : A) (h : R a b) :
  ReflClosure R a b := by
  left      -- 왼쪽 R a b 를 택함
  exact h
```

lean

```
-- 대칭폐포는 대칭관계이다
example (R : A → A → Prop) : Symm (SymmClosure R) := by
  intro a b h      -- h : R a b ∨ R b a
  cases h with
  | inl hab => right; exact hab
  | inr hba => left;  exact hba
```

lean

cases ... with 는 ' $A \vee B$ ' 가정을 두 경우(inl, inr)로 나눈다. 각 경우마다 목표의 알맞은 쪽을 택해 닫는다.

Python — 와샬 알고리즘 구현

와샬 알고리즘은 세 겹 반복문으로 간결하게 구현된다. 점화식을 그대로 코드로 옮긴다.

```
def warshall(M):  
    n = len(M)  
    W = [row[:] for row in M]      # 관계행렬 복사 (W_0)  
    for k in range(n):             # 경유지 k 를 하나씩 허용  
        for i in range(n):  
            for j in range(n):  
                #  $W[i][j] \vee (W[i][k] \wedge W[k][j])$   
                if W[i][k] and W[k][j]:  
                    W[i][j] = 1  
    return W                        # W_n = 추이폐포  
  
M = [[0,1,0,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]  
for row in warshall(M):  
    print(row)
```

python

바깥 반복 k 가 경유지를 늘리는 단계이다. 행렬 W 한 벌을 제자리에서 갱신하므로 메모리가 절약된다.

Python — 세 폐포를 코드로

반사 · 대칭 폐포는 합집합 한 번으로, 추이폐포는 와샬 알고리즘으로 구한다.

```
def reflexive_closure(A, R):          #  $R \cup I_A$ 
    return set(R) | {(a, a) for a in A}

def symmetric_closure(R):             #  $R \cup R^{-1}$ 
    return set(R) | {(b, a) for (a, b) in R}

def transitive_closure(A, R):        # 와샬로  $t(R)$ 
    t = set(R)
    for k in A:
        for i in A:
            for j in A:
                if (i, k) in t and (k, j) in t:
                    t.add((i, j))
    return t
```

python

transitive_closure 는 순서쌍 집합 위에서 와샬 점화식을 직접 적용한 형태이다. 동치폐포는 reflexive → symmetric → transitive 순으로 호출하면 된다.

AI 응용 — 추이폐포와 도달 가능성

추이폐포는 '닿을 수 있는 모든 것' 을 한 번에 알려 준다. 이 성질은 AI 시스템의 여러 곳에서 핵심이 된다.

지식 그래프 추론

'A 는 B 의 상위 개념' 관계의 추이폐포로, 멀리 떨어진 개념 사이의 함의를 모두 도출한다.

멀티홉 RAG

질의 개체에서 추이폐포로 도달 가능한 사실 전체를 후보로 모아, 여러 단계 추론 질문에 답한다.

의존성 분석

코드 · 패키지의 의존 관계 추이폐포로 '간접 의존성' 까지 모두 찾아 빌드 순서를 정한다.

최단 경로

와샬의 사촌 플로이드-와샬은 모든 쌍 최단 거리를 구해 경로 탐색 · 추천에 쓰인다.

5.6 요약 — 연결관계와 와샬 알고리즘

폐포

성질 P 를 만족하며 R 을 포함하는 가장 작은 관계.

반사 · 대칭폐포

$$r(R) = R \cup I_A, \quad s(R) = R \cup R^{-1}.$$

추이폐포

$t(R) = R^\infty$. 모든 지름길을 추가한 관계.

와샬 알고리즘

경유지를 하나씩 허용. n^3 으로 추이폐포 계산.

동치폐포

반사 $\rightarrow$ 대칭 $\rightarrow$ 추이 순으로 적용.

5장 마무리 — 관계의 큰 그림

5장은 '순서쌍'이라는 가장 작은 벽돌에서 출발해, 관계 · 경로 · 성질 · 폐포까지 쌓아 올렸다.

증명

전체 흐름:

곱집합(5.1) → 관계와 세 가지 표현(5.2) → 경로와 연결관계(5.3)
→ 관계의 성질과 동치관계(5.4) → 역 · 여 · 합성관계(5.5)
→ 폐포와 와살 알고리즘(5.6)

관계행렬과 부울곱은 5.3 부터 5.6 까지 모든 계산을 떠받치는 공통 도구.

다음 장으로

7장 부분순서관계는 5.4 의 반사 · 반대칭 · 추이를 갖춘 관계를 다룬다. 5장의 동치관계가 '같음' 을 일반화했다면, 7장은 '순서' 를 일반화한다.

마무리 점검

관계행렬 하나가 주어졌을 때, 다섯 성질을 모두 판별하고 추이폐포까지 구할 수 있는가?